STEP File Tokenizer for Transformer-based Annotation

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Computer Science
Faculty of Engineering and Technology
Sampoerna University
Jakarta
2026

STEP File Tokenizer for Transformer-based Annotation

UNDERGRADUATE FINAL YEAR PROJECT

Hans Sebastian
2022390007

Supervisor 1:	Supervisor 2:
Panji Darma. Ph.D	Supervisor 2 Name with title

May 3, 2026

EXAMINER BOARD APPROVAL

Name of Student	: Hans Sebastian
Student ID Number	: 2022390007
Study Program	: Computer Science
Faculty	: Faculty of Engineering and Technology
Final Year Project Title	: STEP File Tokenizer for Transformer-based Annotation

has been successfully defended in front of the Board of Examiner in partial fulfillment of the requirements for Bachelor of (title degree) in Computer Science, Faculty of Engineering and Technology, in Jakarta, May 3, 2026.

Board of Examiners

Examiner 1

(wet ink signature)

Examiner 1 Full Name with title

Examiner 2

(wet ink signature)

Examiner 2 Full Name with title

Supervisor 1

(wet ink signature)

Panji Darma. Ph.D

Supervisor 2

(wet ink signature)

Supervisor 2 Name with title

DECLARATION OF ORIGINALITY

I, Hans Sebastian 2022390007, here acknowledge that this submission entitled "STEP File Tokenizer for Transformer-based Annotation" is my own work with the guidance from my supervisor. I also truly acknowledge that it contains no materials previously published or written by another person, or any other educational institution, except where due acknowledgment is made in the Final Year Project. Any contribution made to the research by others is explicitly acknowledged in the Final Year Project.

Jakarta, May 3, 2026

Hans Sebastian

EXCLUSIVE RIGHT STATEMENT

I, Hans Sebastian 2022390007, hereby grant to our school, Sampoerna University, the non exclusive right to archive, reproduce, and distribute my Final Year Project entitled “STEP File Tokenizer for Transformer-based Annotation” in whole or in parts, whether in printed or electronic formats. I acknowledge that I retain exclusive rights of my Final Year Project by using all or parts of it in future work or output, such as articles, books, software, and information system.

Jakarta, May 3, 2026

Hans Sebastian

ACKNOWLEDGEMENT

The completion of this project cannot be realized without the help of my supervisor, Panji Darma. I offer my greatest appreciation for the guidance and lessons you gave regarding my writing.

Jakarta, May 3, 2026

Hans Sebastian

ABSTRACT

Full Name : Hans Sebastian

Study Program : Computer Science

Title : STEP File Tokenizer for Transformer-based Annotation

Supervisor : Panji Darma. Ph.D

Current CAD asset management is hindered by manual annotation and automated methods relying on lossy visual conversions or unavailable construction histories. This study proposes a novel Transformer-based architecture for the direct hierarchical annotation of raw Boundary Representation (B-Rep) STEP files. We introduce a Hierarchical Dual-Stream Tokenizer utilizing Semantic Macro-Compression to reduce sequence complexity, Canonical Normalization via PCA for rotational and translational invariance, and deterministic Topological Linearization.Leveraging a Qwen 3.5 backbone, the architecture employs Dual-Stream Encoding to fuse discrete semantic tokens with complex-valued geometric embeddings via Geometry-Infused Attention and Rotary Positional Embeddings (RoPE). Following the LIMA principle, the model was fine-tuned on $N = 2,500$ high-fidelity, human-verified CAD files.Results demonstrate robust performance across a three-tiered schema: Level 1 functional classification (BERTScore: 0.95), Level 2 structural description (ROUGE-L: 0.88), and Level 3 parametric precision via constrained decoding (BLEU-4: 0.85). Inter-rater reliability was successfully validated using Cohen's Kappa. This scalable framework effectively captures functional identity and precise parametric constraints without requiring procedural histories, revolutionizing industrial CAD management.

Keywords: CAD, STEP, B-Rep, Transformer, Dual-Stream Tokenizer, Semantic Macro-Compression, PCA, RoPE, BERTScore, LIMA, inter-rater reliability, ROUGE-L, BLEU, Cohen's Kappa.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF FORMULAS

CHAPTER I

INTRODUCTION

1.1 Background

1.1.1 Blueprinting and Other Traditional Copying Methods

Before digital technology became widespread, engineers, architects and designers used conventional methods such as mylars and blueprints which require the drawings and measurements to be copied manually by dozens of workers. Blueprints utilized chemicals such as potassium ferricyaniide and ferric ammonium citrate solution to coat the original drawing to be copied. The coated paper is then exposed to ultraviolet light, creating a white-on-blue copy after washing it with water (Herschel, 1842). This method suffers from 2 main drawbacks which limits productivity and flexibility when designing. One such drawback is the modification issue, which is that the master design to be fixed before copying, meaning any changes to the original causes the copies to be deprecated and the entire process redone from scratch. The other issue comes with the need to dry the copies before they become usable, creating more overhead.

Whiteprinting attempted to replace the original blueprinting technique by utilizing ammonia fumes instead of water, resulting in a black-on-white copy that does not need to be dried (Parmeggiani, 2014). Although it improved efficiency by creating a dry copy, whiteprinting includes the creation of hazardous fumes such as ammonia which could cause long-term damage to the operator (McGlothlin, 1981). Furthermore, this technique still suffer from the modification issue that plagued the original. These downsides pushed engineers to develop better, more efficient alternatives.

1.1.2 Computer Aided Design

Advances in technology have allowed for the development of better procedures, especially computer aided design (CAD) which largely alleviated the issues present in prior methods. CAD can be defined as the use of digital computers to create, modify, analyze, and optimize existing designs (Sadrehaghighi, 2022). In other words, CAD is the process of utilizing digital technology to perform the entire design process, from ideation to generation and sharing, bypassing the need for traditional copying procedures such as blueprinting. The concept originated from the first program *Sketchpad* designed by Ivan Sutherland in 1963 (Sutherland, 1964), and has received numerous improvements with newer tools and programs such as SolidWorks (Zeid, 2021). A short diagram detailing the CAD process can be seen in Figure 1.

In the current era, the use of CAD tools and software have largely replaced such methods as they lack the limitations that come with traditional techniques. This is especially true in engineering-related fields

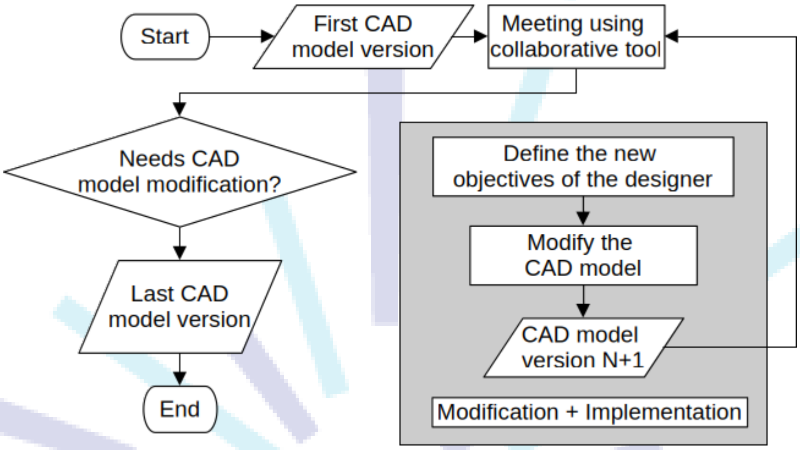

Figure 1 Flowchart of the CAD modeling process (Fleche et al., 2014)

with one study showing up to 45–60% reduction in task duration (Rahman et al., 2019; Babayev and Mammadli, 2025). Other benefits include the ability for immediate changes even during late-stage development which helps limit delays (Aranburu et al., 2021). The main benefit of CAD tools is ability to reutilize/repurpose existing CAD designs to be reused in other CAD projects, resulting in more efficient, reliable workflows that eliminates the need to redesign already existing components from scratch (Gutiérrez de Ravé et al., 2025).

Although the use of CAD have improved the overall design and manufacturing process, they also pose their own set of issues. For example, CAD software developers often employ their own proprietary file formats, each with their own complex structures. This results in major incompatibility issues as users might have different tooling preferences. As a result, international governing bodies such as International Organization for Standardization (ISO) have developed universal CAD file formats such as Standard for the Exchange of Product Data (STEP) to handle the issue of compatibility. A STEP file consists of components that each describe geometric data using a language called EXPRESS (Pratt, 2001). Another universal CAD file format include STL designed by 3D Systems in 1987, which contains a set of triangles consisting of facets and vertices (Szilvsi-Nagy and Mátyási, 2003).

Another problem comes with difficulties in documentation and indexing for existing CAD designs, as the complexity of the file's structure inhibit the use of automated annotation and indexing tools, forcing the use of manual annotation. This procedure, although viable for small designs, quickly becomes time consuming and error-prone as the size of a project grows. Although the universal file formats help with compatibility between different CAD software, issues regarding documentation and indexing still persists, prompting the research for potential solutions.

1.2 Problem Formulation

There are multiple issues relating to CAD design annotation, one example being that manual annotation incurs an unnecessary overhead, diverting manpower and time away from the main design process. Furthermore, manual annotation also introduce risks regarding annotation quality and accuracy, as human annotators might miss important details or mislabel important components, reducing overall workflow efficiency (Mandelli and Berretti, 2022). Difficulties in determining annotation formats and standards also increases risk of errors, which is especially noticeable when dealing with third-party designs.

The development of artificial intelligence (AI) have resulted in new techniques for automating the process, however most AI-based methods and models require the use of additional conversion steps into images and/or point clouds to leverage existing image processing and computer vision techniques (Mandelli and Berretti, 2022). This added steps reduce the overall efficiency and increases the risk of misclassification. Additionally, current state-of-the-art (SotA) models such as Gemini 3 and Chat-GPT 5.2 overly rely on the existing metadata contained in the file, completely bypassing the geometry of the design. Other issues related to these models include privacy and latency concerns due to the necessity for an internet connection, as well as high computational and energy costs as the models are designed for general use (O'Donnell and Crownhart, 2025). This study aims to develop a novel method for processing CAD files which preserves the existing semantic context and connections between the parts without the need for additional conversion steps into visual representations.

1.3 Research Objectives

Based on the topic discussed in the Problem Statement section, this study aims to complete the following research objectives:

1. Develop a Transformer-based system capable of generating high-fidelity functional descriptions for unlabelled STEP files, validated against human-authored ground truth using BERTScore.
2. Translating low-level B-Rep topology into high-level semantic concepts with $> 80\%$ identification accuracy as measured with BERTScore and BLEU measures.
3. Enhance CAD file indexing by leveraging model-generated descriptions as semantic metadata.

1.4 Significance of Study

This study's significance and contributions towards existing literature are detailed as follows:

1. Enable faster development and project organization with automatic STEP CAD file annotation.
2. Develop an precision-preserving parsing method for **Boundary Representation** (B-Rep) data in STEP files.
3. Introduce a **Hierarchical Dual-Stream Tokenizer** that converts complex 3D connections into a deterministic sequence, enabling the Transformer to consistently identify parts regardless of file scrambling (R. Wu et al., 2021).

1.5 Limitation of Study

The following study lists the following limitations and potential improvements:

1. **Context-dependence:** The labeling of CAD objects relies on designer intent, which might be lost during pre-processing.
2. **Implicit Bias:** The correctness and quality of the output and training process are determined based on human responses, which introduces potential bias.
3. **Limited Vocabulary:** The model's output and training data are based solely on the English language, limiting its ability to generate annotations in different languages.
   Additionally, the model's vocabulary is modified to reduce subjectivity by limiting the use of adjectives such as "big" or "small".
4. **Sparse classes:** The model's training data only consists of general common objects which can be easily found in public datasets in order to maximize the accuracy of the model. This is due to the scarcity of complex CAD designs in such datasets as they contain mostly reusable assets.

1.6 Organization of the report

The rest of the paper will be organized according to the following structure: Chapter II analyses the existing literature and basic terminologies required to understand the topic. Chapter III details the methodology used to gather the training dataset and the design considerations related to model development, from tokenizing, fine-tuning, to the benchmarking process. Chapter IV lists the results gathered during the benchmarking process paired with detailed analysis of said results. Lastly, Chapter V summarizes the conclusion obtained from the study as well as potential improvements for future research.

CHAPTER II

LITERATURE REVIEW

2.1 Data and Information

2.1.1 Definitions

Data refers to a collection of raw, unlabelled information which represents a set of truth/facts about the world (Olson, 2021). Data comes in two main categories, quantitative and qualitative (Hackman et al., 2024). Quantitative data refers to data which has a distinct form, with unique categories and can be analyzed using objective numerical operations and methods, where qualitative data lacks a concrete structure and is highly context-dependent and subjective (Schoonenboom, 2023). Quantitative data mostly consists of values that can be represented numerically (a person's height/weight, price of an object), while qualitative data refers to data about a subject such as nominal descriptions (a person's name, a novel's synopsis). This paper focuses on digital data, which is data represented as a sequence of symbols, mainly binary digits.

The concept of data differs from information in that data serves as the foundation from which information is generated (Hackman et al., 2024). In essence, information is a specific interpretation of a given dataset, derived from their type and values, as well as their context and semantic meaning. In particular, the context in which a collection of data is acquired determines the set of valid interpretations of said data, which in turn affects the possible inferences i.e. potential information gathered from it (Mason, 2019). Raw data in itself cannot be used directly, as it merely represents the attributes of a certain subject, without assigning to it any valid implication/insight. As such, the use of data involves converting it into information through analysis and contextualization (Borrohou et al., 2025).

As the name suggests, information is the basis of all modern information technologies and systems such as search engines, recommendation systems, and artificial intelligence. These systems utilize information gathered from large datasets to gain knowledge, derive solutions and make decisions such as giving a custom-tailored recommendation to particular set of users in the case of recommendation systems, and generating responses in the case of AI. The amount of data generated daily has grown exponentially in the information age, with one estimate stating approximately 402.74 million terabytes are generated by electronic devices around the world daily, and an estimated 394 zettabytes will be generated by 2028 (Taylor, 2025). This prompts the need to develop a method for organizing, analyzing, and managing large batches of data efficiently and accurately.

2.1.2 Data classification

Data classification is the process of assigning labels to data for better management purposes (Newhouse et al., 2023; Shivayogi, 2025). In short, data classification is the process of grouping data into different categories according to their characteristics for later processing and bookkeeping. Data classification allows for enhanced protection, security and risk management of data based on sensitivity and regulatory standards (Lane and Adelusi, 2023; Nguyen and Cuong, 2025; Newhouse et al., 2023). Furthermore, data classification allows for better organization and retrieval of certain types of data, improving the accuracy and performance of information gathering. Data classification methods largely follows this specific sequence of steps: 1) defining classification criteria, 2) data discovery and sensitive data detection and 3) data labeling based on type, usage, and security requirements (Shivayogi, 2025).

Most forms of data, such as text, images, and video/audio recordings, are categorized as qualitative as they lack a formal, fixed set of attributes and structure. Furthermore, data with complex structures such as CAD files contain a large amount of features and attributes that form deep semantic relationships. As a result, these types of data cannot be easily classified using conventional methods as they rely on assumptions regarding the data's structure. Recent advancements in artificial intelligence systems such as natural language processing and deep learning have allowed for reliable, efficient classification of these datatypes (Blessing, 2021). In modern systems, vector embeddings have become the cornerstone that enables these capabilities, replacing the less efficient rule-based approach in natural language processing, and acting as the basis for deep learning systems such as convolutional/recurrent neural network models (Patil et al., 2023; Asudani et al., 2023). This paper will focus on the use of vector embeddings as a method for enumerating and processing CAD files for classification purposes.

2.2 Vector Embeddings

2.2.1 Definition and Purpose

Vector embedding is defined as a method for converting and representing complex, unstructured data as a sequence of numerical values (Pajo, 2025; Chitturi, 2024). A vector embedding aims to create a compact, structured representation of an underlying datapoint while preserving the underlying context and semantic meanings contained in it (Chitturi, 2024). In other words, vector embeddings encode unstructured, context-rich data into points in an n-dimensional space, capturing the relationships of different data points in numeric form, allowing the data to be processed and analyzed using mathemathical operations (Pajo, 2025). This representation also renders the concept of "distance" as a quantifiable component in data analysis, where distance refers to semantic proximity/closeness in meaning (Kukreja et al., 2023). In short, the distance between n-dimensional vector representation of two datapoints closely models the semantic relationship between the original data.

Vector embedding serves as a way to efficiently encode large datapoints such as texts, images, video and audio into a digestible, compact format that could be easily processed by other systems. Vector embeddings also allows data to be searched, indexed, and grouped using their semantic similarity (Taipalus, 2024). One use case is in large language models, where models could be equipped with the ability to access external documents and data, allowing it to produce reliable output grounded on falsifiable truth (Omolayo and Babatope, 2025).

Other similar methods for encoding qualitative, complex data have been developed, such as one-hot encoding, which encodes each datapoint as states represented as a sequence of n-bits, where all but one bit is set to '0', and the '1' bit represents the current state of the system (Samuels, 2024). Another example is Bag of Words (BoW), which represents text as a "bag"; an unordered, unstructured collection of its words, or keypoints in the case of an image (Qader et al., 2019), and Term Frequency-Inverse Document Frequency (TF-IDF), which encodes words/keypoints as its number of occurrence and rarity inside a document (Qaiser and Ali, 2018).

Nonetheless, the methods listed previously face several limitations when compared to newer techniques such as vector embedding, such as the loss of semantic information such as the word order, grammatical structure, and other relevant context stored in the original. Other problems include the problem of dimensionality, which states that as the number of dimensions increase for a given set of data, the amount of available data becomes increasingly sparse and patterns more obscure (Banks and Fienberg, 2003). This limitation also massively increases the compute power and memory required to store and process these encodings while also increasing the risk of errors (Trunk, 1979). Vector embedding sidesteps these limitations by mapping similar items to vectors which have a close distance to each other, preserving semantic data and minimizing the number of features required. The shift from sparse, count-based encoding methods towards a more compact, learned encoding technique enables critical performance improvements in terms of data processing metrics such as compute time and output accuracy.

2.2.2 Embedding Models

Although vector embedding offers several important advantages, such advantages require specialized, novel machine learning models that are capable of distilling complex relationships between datapoints into a compact dense vector representation while minimizing the amount of features/dimensions. These models capture context ingrained within datapoints by learning how to assign a connection between two neighboring words/parts of data (Tran, 2024, Chapter 4.1).

Most embedding models could be categorized into two groups: static models that generate a single, fixed representation for a word/data segment, such as continuous BoW and Subword models (Wang et al., 2019), and contextual models such as Bidirectional Encoder Representations from Transformers (BERT) and its variants that generate unique representations of the same word depending on the current context (Liu et al., 2025; Devlin et al., 2019). In recent times, contextual models have largely dominated due to their flexibility and better capabilities in terms of capturing the nuance and implicit interconnections between datapoints and its components. One popular example of this is the transformer architecture, which forms the basis the paper's research.

2.3 Transformer Architecture

2.3.1 Architecture Overview

Current state-of-the-art (SotA) embedding models are based on the transformer architecture, which acts as the foundation for commercial generative AI models such as ChatGPT, Gemini, and DeepSeek. Introduced in the seminal paper "Attention Is All You Need" (Vaswani et al., 2017), instead of processing data sequentially, the architecture utilized a novel approach for sequential, recurrent processing tasks referred in the paper as **Self-Attention Mechanism**, replacing the recurrent, in-order structures of previous models like recurrent neural networks (RNNs) and long short-term memory models (LSTMs) (Sherstinsky, 2020; Hochreiter and Schmidhuber, 1997). This mechanism allows the model to process input sequences in parallel, which is critical for handling non-local graph structures present in STEP files.

Central to this architecture is **Tokenization**, a process of dividing input data into smaller chunks referred to as tokens. The tokens are represented as numeric vectors, reducing the amount of storage and compute needed for further processing. The literature dictates two main paradigms of tokenizing input data, full word and subword tokenization. Full word tokenization, as it implies, splits input data based on a pre-defined separator, such as whitespace in text. The above method have largely been replaced by its counterpart, which divides input data into smaller subparts that has no fixed separator. Examples of subword tokenization include Byte-Pair Encoding (BPE) and WordPiece (Suyunu et al., 2024; Kozma and Voderholzer, 2024; Devlin et al., 2019). The tokens generated from the process are then used as the main input data of the self-attention mechanism. While traditional models utilize subword tokenizers like BPE to parse natural language, this research adapts the concept to the geometric domain. Just as an LLM tokenizes words to understand a sentence, the proposed architecture tokenizes topological entities (e.g., surfaces, curves) to "read" a 3D object. These geometric tokens are then processed by the attention mechanism to generate semantic descriptions.

The self-attention mechanism detailed in the transformer architecture improves upon previous systems by enabling models to evaluate the importance of the surrounding parts of an input sequence, regardless of its ordering. For instance, consider the sentence “The dog barked at the cat to drive it away.” Using self-attention, transformer-based models can correctly associate the word “it” with the word “cat” rather than “dog” (Vaswani et al., 2017). This capability of handling remote dependencies and understand context allows it to outperform previous architectures. The details of the mechanism lies in the novel components that make up the architecture, forming a deep network. In particular, the transformer model’s functionality and reliability is the outcome from combining two key components, encoders and decoders. Figure 2 details the architecture of a standard transformer.

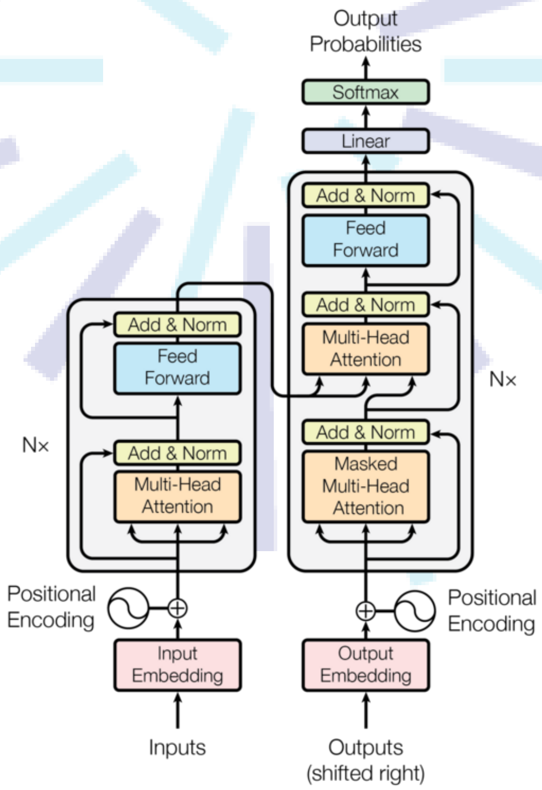

Figure 2 Standard Transformer Architecture consisting of an Encoder and Decoder (Vaswani et al., 2017)

2.3.2 Encoder-Decoder

The encoder-decoder components consists of several blocks/subcomponents that in turn form the basis of a transformer layer, which are then stacked continuously to build the complex models. The encoder functions as a processing layer for the entire input sequence, aiming to build a rich, contextualized numerical representation utilizing the self-attention mechanism. The encoder is implemented as two main parts:

1. **Multi-Head Self-Attention:**
   The self-attention of the transformer is employed as different units/heads

working in parallel, allowing the model to learn different types of relationships from the input data simultaneously (Voita et al., 2019). Each "head" is tasked with learning a different aspect of the given sequence. For instance, one head might focus on syntactic relationships while another learns semantic ones. This functionality is done through projecting the input data into different representation subspaces and applying self-attention in parallel. The final result is a vector generated by applying a linear transform to the concatenation of the heads' output (Vaswani et al., 2017)

2. **Position-wise Feed-Forward Networks (FFN):**
   The next step consists of passing the token vectors gathered and weighted from the attention mechanism through a feed-forward neural network. The subcomponent consists of two linear neural network layer with a non-linear activation function in between, such as ReLU. It enriches each token's representation by first expanding the vector's dimension (i.e. 512 to 2048), applying an activation layer and resizing the vectors back into its original size. This process allows the model to extract deep, interconnected patterns hidden within the data. Furthermore, the network acts as a key-value store for patterns learned during training, and its output represents a distribution of tokens containing said patterns (Geva et al., 2021).

The parallel nature of self-attention makes it difficult to retain information regarding sequence ordering. Hence, positional encodings in the form of special vectors are added to the embedddings to mitigate this limitation. The positional encoding vectors are generated using sinusoidal functions with different frequencies that represent positions of tokens inside the sequence (Vaswani et al., 2017). After the encoding process, the result generated by the encoder is passed through a decoder that remaps the resulting vectors back into tokens that forms the final output sequence. The decoder achieves this through three steps:

1. **Masked Multi-Head Self-Attention:**
   The first step of the decoding sequence is similar with encoding, with a key difference being he lack of a lookahead for future tokens. This is done to prevent model overfitting as knowing future tokens might embed irrelevant connections into the model.

2. **Cross-attention:**
   Another change made in the decoding process is the addition of a cross-attention layer. The layer acts as a critical link between the encoder and decoder that functions as a window for the decoder to look at the encoder's final output. The encoder uses its internal state as a query in the form of a vector and relates it to the vector set generated by the encoder. This process enabels the decoder to determine the most relevant parts of the input and its relation to the final output (Vaswani et al., 2017).

3. **Position-wise Feed-Forward Networks (FFN):**
   The final decoding step is similar to the encoding process with no changes.

Enabling effective training of densely-layered networks require proper integration of the encoder and decoder. These requirements prompts the use of two operations which are applied post-encoding and decoding to ensure stability and effective training. The integration involves adding a residual connection layer that adds the original input vector with the output to minimize the vanishing gradient problem. Next, the vectors are normalized and stabilized, ensuring a smoother and reliable training process. The architectural decisions that make up the model allows the connections between each token to be filtered and strengthened based on the input's context, and allows the model to retain more information from its training data (Vaswani et al., 2017).

2.4 Related Works

Early attempts at classifying CAD files mainly used feature descriptors such as Zernike and Reeb graph descriptors to determine the category of a CAD model based on shape and function (Ip and Regli, 2005). Several other works focuses on 2D based methods such as utilizing machine learning to classify over 1600 CAD files using random forest, support vector and fully-connected neural network models by identifying 45 different basic shapes (Machalica and Matyjewski, 2019), with subsequent approaches combining GNN to measure and group CAD files based on their similarities in geometry (Roj et al., 2021). More examples of this type include RotationNet, which utilizes images of the same 3D objects from different angles to estimate its pose and category (Kanezaki et al., 2018).

Different category of related studies uses a 3D-based approach, with one leveraging the use of 3D grid representation (voxels) of the CAD file and applying a 3-dimensional-CNN that could detect and categorize features (Maturana and Scherer, 2015), and another applies a Mask-recurrent CNN model to extract 3D objects from a single 2D RGB image which can then be classified (Gümeli et al., 2022). Works such as PointNet utilizes 3D point clouds to estimate the class of an object (Qi et al., 2016). A different classification uses a custom 3D representation named PANORAMA and feeding the result to a CNN to find keypoints and classify the CAD based on the detected features (Papadakis et al., 2009; Sfikas, Theoharis, et al., 2017; Sfikas, Pratikakis, et al., 2018). Newer 3D-based approaches utilizes graph-based techniques to make use of the graph-like structure of CAD files such as STEP, with one work using Graph-based Neural Network (GNN) to categorize certain CAD STEP files into 7 different classes (Mandelli and Berretti, 2022). However, a research gap in previous works can be identified, since the cited works are limited to classifying a fixed class of CAD objects, such as bolts, and cannot be used to create a general description of objects not specified in the class dataset.

Another method applies a Feature Directed Acyclic Graph (FDAG) to determine the dependency between the shapes of CAD models which are then optimized until only the fundamental features and shapes remain. The shapes can then be compared to CAD models to determine their resemblance to the shapes used as a query (M. Li et al., 2011). One graph based method also utilizes the structure of STEP files to generate an attribute graph which can then be used as a measure for shape similarity (El-Mehalawi and Allen Miller, 2003a; El-Mehalawi and Allen Miller, 2003b). However, while these graph-theoretic approaches excel at topological retrieval, they remain disconnected from the semantic intent of the design.

To bridge the gap between geometric topology and natural language, recent frameworks have turned to multi-modal learning. Text2CAD provides a seminal contribution by pairing natural language with CAD geometry, its methodology relies entirely on Programmatic Synthesis (Khan et al., 2024). The dataset is constructed by first generating CAD scripts and then using Large Language Models (LLMs) to reverse-engineer textual descriptions from the code. Nevertheless, the approach introduces a research gap as real-world industry STEP files (ISO 10303-21) rarely contain the clean construction history (e.g., Extrude, Revolve) found in Text2CAD. Figure 3 details the overall flow of Text2CAD. They are often fixed Boundary Representations (B-Reps) containing only explicit surfaces and topology. Therefore, a model trained solely on Text2CAD fails to interpret the vast majority of preexisting CAD files. Moreover, the research focuses on generating the step-by-step geometric operations required for reconstruction (e.g. "Extrude by 2cm"). While this results in precise parametric descriptions, the model effectively describes the object's recipe without understanding what the object is. For instance, while it may generate highly accurate construction steps for a set of interlocking toothed cylinders and shafts by specifying exact relative measurements for every extrude and cut, it frequently fails to recognize or categorize the resulting assembly by its fundamental functional identity: a Gearbox. Consequently, the model provides the fabrication steps but lacks the capability to semantically categorize said object.

To overcome the dependency on synthetic construction history and the lack of functional understanding, this study introduces a tokenizer and annotation pipeline designed explicitly for Reconstruction-Free B-Rep Graphs. Furthermore, to ensure the model captures the high-level semantic identity of components rather than just their geometric recipe, this research employs a Human-in-the-Loop (HITL) protocol to directly author and verify the dataset. This verification process guarantees that the descriptions align with the visual reality of the part, prioritizing functional categorization over procedural fabrication.

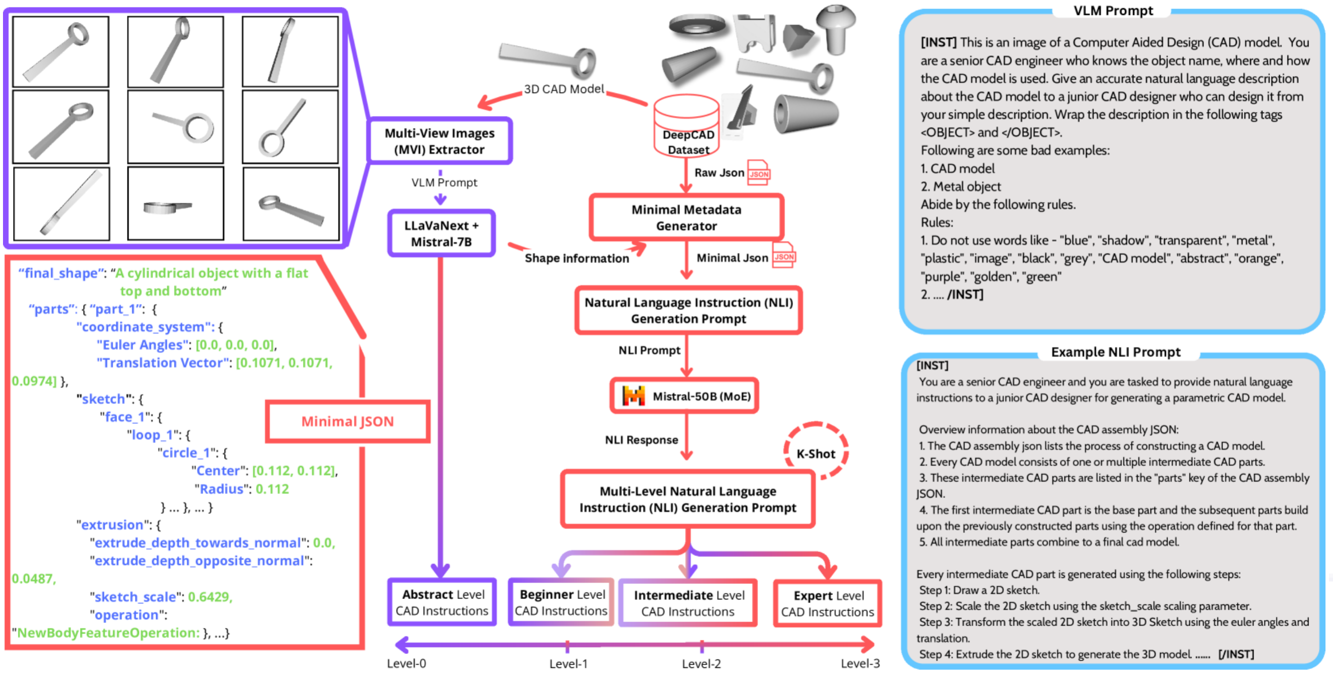

Figure 3 Text2CAD Annotation Generation Pipeline (Khan et al., 2024)

CHAPTER III

METHODOLOGY

3.1 Data Gathering

Training a descriptive model effectively necessitates a large-scale, high-fidelity dataset that pairs geometric representations with semantic natural language. However, unlike 2D image-captioning domains, the 3D CAD landscape suffers from a significant scarcity of standardized, human-readable annotations. A comprehensive data acquisition framework was developed to handle the sparse data availability. The procedure involves ingesting raw geometric data, filtering for quality and file size, and semantically enrich samples through a hybrid HITL process. A summary of the dataset preprocessing steps can be seen in Figure 4.

3.1.1 Dataset Source Selection

Publicly available datasets for CAD file designs can be found on the internet. One prominent example is the ABC dataset, which consists of over one million unannotated STEP files derived from Onshape (S. Koch et al., 2019). Another is the Fusion 360 Gallery, a dataset of free user-submitted designs with design timeline (Karl et al., 2021), and Text2CAD, which hosts around 170,000 CAD models with 660,000 natural language annotations ranging from short descriptions to detailed explanations (Khan et al., 2024). Other sources include free 3D/CAD design websites such as GrabCAD and TraceParts, which contain various CAD objects not found in compiled datasets. A comparative summary for the sources listed above is contained in Table 1 below.

Table 1 Comparison of CAD Dataset Sources.

Source	Type	Summary	Advantages	Disadvantages
TraceParts	Online Library	MFR-verified standard parts (bolts, gears, bearings).	High-fidelity geometry; ISO/DIN compliant.	Limited to standard hardware; requires scraping.
GrabCAD	Online Library	6M+ user files	High diversity of complex/creative user-made designs.	Inconsistent quality; non-manifold geometry.
Text2CAD	Dataset	Annotated CAD files with text-geometry pairs.	Ready-to-use labels; strictly organized.	Lower complexity than industry files.
Fusion 360	Dataset	Designs exported with assembly timelines.	Includes reconstruction history (timelines).	Proprietary format; inconsistent exports.
ABC dataset	Dataset	1M+ unlabelled B-Rep files from Onshape.	Massive scale; robust topology for pre-training.	No semantic labels; contains "garbage" data.

Based on the analysis detailed in Table 1, this paper will utilize a combination of CAD designs contained in the ABC dataset as well as other online sources such as TraceParts. This combination ensures the dataset contains a balance between real-world user designs (ABC) and high-definition examples that adhere to industry standards (TraceParts). To generate standardized annotations from several different datasets sources, a standard for annotation format mimicking the existing annotations in the Text2CAD dataset will be used.

3.1.2 Dataset Preprocessing

3.1.2.1 Dataset Filtering

Before training, the dataset undergoes a rigorous analysis to eliminate rare or low-quality CAD designs that could destabilize the model. The filtering process is divided into two distinct stages. The first stage is **Class and Format Standardization**, implemented using PythonOCC. The script scans the raw data to enforce domain relevance by discarding any files not listed in the target object whitelist (e.g., excluding rare CAD designs and broken/corrupted files). Valid JSON-based CAD files are then automatically converted to STEP format using the OpenCascade library, while other unrecognizable formats are purged from the dataset.

The next stage focuses on **Manifold Validation**. Since raw STEP files frequently contain disjointed surfaces or unstitched faces, a "watertight" topological check is enforced, Using volumetric analysis (Volume

> 0), any file containing open shells or non-manifold geometry is explicitly discarded. This initial geometric sanitation prevents invalid outliers from polluting the training signal, ensuring that the model only learns from physically realizable 3D objects.

The second stage addresses statistical robustness by identifying and pruning *long-tail* categories, or more specifically, classes containing fewer than 100 unique geometric instances. This threshold is critical for preventing model overfitting. Deep learning models often resort to memorization rather than generalization when handling classes with insufficient sample sizes (Goodfellow et al., 2016). As demonstrated by C. Zhang et al. (2025), neural networks possess the capacity to fit arbitrary noise; in the absence of large-scale data constraints, they tend to memorize specific training instances rather than extracting abstract rules. This phenomenon leads to *shortcut learning*, where the model identifies objects via spurious noise patterns or texture statistics instead of meaningful topological features (Geirhos et al., 2020; Arpit et al., 2017).

Furthermore, recent studies indicate that this vulnerability is exacerbated in high-variability datasets. Research by You et al. (2025) demonstrates that machine learning models struggle significantly to identify low-occurrence classes where the "class pattern" breaks due to high intra-class variance. A minimum threshold of 100 instances is enforced during the filtering process, ensuring that every semantic category (e.g., `GEAR`, `BRACKET`) possesses sufficient geometric variance to force the Transformer to learn a generalized representation of the object type, rather than memorizing a handful of outliers.

While modern deep learning often favors massive dataset scale, this research applies the **Less Is More for Alignment (LIMA)** principle (Zhou et al., 2023). The LIMA framework posits that if a base model possesses sufficient generalized intelligence, domain adaptation requires data *quality* and *density* rather than sheer volume.

Consequently, instead of brute-forcing the model with 100,000 noisy, unverified CAD files, the dataset is aggressively filtered down to a highly curated, ultra-high-fidelity subset of $N = 2,500$ geometrically dense STEP files. Because each file is unrolled into a complex multi-node graph, these 2,500 files contain millions of distinct geometric relationships. By training exclusively on verified, high-quality data, the model converges on the complex mapping between Stream B mathematics and Stream A semantics without memorizing the spurious noise and hallucinated annotations prevalent in scraped datasets.

3.1.2.2 Dataset Standardization and Annotation Format

The next preprocessing step involves designing an annotation standard for the unannotated CAD dataset sources to ensure homogeneity with the pre-annotated Text2CAD dataset through a multi-stage standardization and formatting pipeline. The pipeline standardizes varied input structures into a consistent, unified representation matching the multi-level description schema of the Text2CAD dataset.

Data sourced from TraceParts often contains descriptive but non-stand -ardized filenames (e.g., *"Hexagon_Head_Screw_ISO4017_M8.stp"*). To extract the functional class, a **Hierarchical Regex Parser** is employed. The parser utilizes a dictionary of domain-specific keywords to map CAD files into general, high-level categories to ensure accurate annotation. The mapping logic follows a precedence order specified below to resolve ambiguities:

1. **Standard Extraction:** Strings containing ISO/DIN codes (e.g., "ISO 4017") are mapped immediately to their corresponding standard object family (e.g., `FASTENER`). This is done to exploit the rigorous standardization inherent in industrial engineering; parts explicitly labeled with international standards (ISO, DIN, ANSI) possess a verified, unambiguous functional identity, allowing them to serve as "Ground Truth" anchors for the classifier with near-100% label confidence.
2. **Morphological Keyword Search:** If no standard code is found, the parser scans for morphological roots. For example, filenames containing "gear", "pinion", or "worm" are all mapped to the class `GEAR` to simplify the manual annotation step.
3. **Exclusion Logic:** To prevent misclassification of assemblies, filenames containing "Assy" or "Assembly" are flagged and excluded from the single-part classification dataset, as assembly CAD files s inherently represent composite systems containing multiple distinct components (e.g., a gearbox comprising shafts, bearings, and housings). Including these files would introduce label ambiguity, as a single input graph would correspond to multiple conflicting functional classes, which is outside the research's scope.

Finally, while the hierarchical schema structure is retained for compatibility, the semantic objective of the annotation fields is redefined. Instead of describing the steps required to generate the object, the multi-level schema is used to categorize annotations of the same object based on their level of detail. Table 2 below shows the defined annotation levels with an example for each level.

Table 2 Proposed Hierarchical Annotation Levels for Parametric Captioning

Level	Scope	Definition	Example Annotation
Level 1 (Abstract)	**Functional Category**	Identifies semantic class and primary application. Useful for high-level retrieval.	"A standard metallic **fastener** designed for high-torque applications."
Level 2 (Concrete)	**Morphological Description**	Describes visual geometry and topological features (shape types, head styles) without specific dimensions.	"A **hexagonal-head bolt** featuring a partially threaded cylindrical shaft and a chamfered tip, with no washer face."
Level 3 (Detailed)	**Parametric Specification**	Provides precise constraints, dimensions, and standard designations required for reconstruction.	"An **ISO 4014 Hex Bolt** with a nominal thread diameter of **8mm** (M8), a total shaft length of **40mm**, and a thread pitch of 1.25mm."

3.1.2.3 Annotation Validation using BERTScore and Inter-Rater Reliability

Since the dataset is comprised of multiple sources, with most lacking the necessary descriptive texts, an additional annotation step is needed. Unlike simple classification, generating natural language descriptions requires high-fidelity textual data that automated processes cannot fully synthesize. To extend the dataset beyond the pre-annotated Text2CAD samples, a **Human-in-the-Loop (HITL)** annotation pipeline is employed. Unlike simple filtering, this process utilizes **BERTScore** (T. Zhang et al., 2020) as a semantic validation metric to guide the manual annotation process, ensuring that new descriptions align stylistically with the established Text2CAD annotation schema specified in Table 2.

The initial step includes writing manual annotations for each unannotated dataset entries which are validated by domain experts. The captions are designed to accurately describe the unlabelled geometric files (e.g., from TraceParts) with varying levels of detail. To maintain consistency, annotators follow a strict syntactic template derived from the Text2CAD dataset e.g. *"[Functional Component] with [Geometric Features] and [Dimensions]."* This ensures that the training data remains uniform across different sources.

To validate the quality of the human-generated descriptions, a blind cross-verification study was conducted. A senior domain expert will be assigned to review a random sample of $N = 100$ annotated descriptions, rating them as either "Compliant" or "Non-Compliant" based on the style guide. The consistency of this quality control process was measured using **Inter-Rater Reliability (IRR)**, specifically **'s Kappa** (κ) (Cohen, 1960). This metric was chosen over simple percent agreement to robustly account for the possibility of chance agreement, with a target threshold of $\kappa > 0.80$ established to certify the dataset's reliability for training the transformer model (Landis and G. G. Koch, 1977). Details of the equation can be seen in paragraph 3.3.2.1.4.

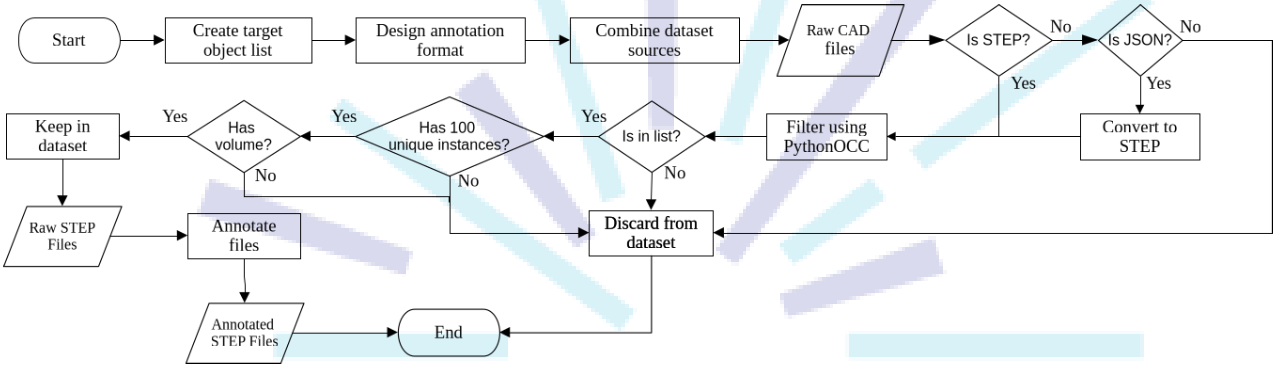

Figure 4 Dataset acquisition and filtering flowchart

3.2 Tokenizer Design

This research proposes a novel *Hierarchical Dual-Stream Tokenizer* designed specifically to address the non-Euclidean nature of B-Rep (Boundary Representation) data. Unlike standard Natural Language Processing (NLP) tokenizers, which treat input as a continuous 1D string under the assumption that positional proximity equals semantic relationship, the proposed method treats the STEP file as a **topological graph**. This distinction is critical, as in raw B-Rep data, geometric entities that are physically adjacent (e.g., two faces sharing an edge) may be defined thousands of lines apart in the file structure. Therefore, **normalization** is required to remove coordinate invariance, and then **compressed** to eliminate redundant entity definitions, and **linearized** via a graph traversal to preserve local geometric neighborhoods before ingestion.

3.2.1 Tokenizer Architecture

The tokenization pipeline consists of four distinct stages designed to ensure geometric invariance and computational efficiency. The overarching goal is to bridge the *semantic gap* between the low-level mathematical definitions of CAD surfaces and the high-level semantic reasoning required by the Transformer.

Standard tokenizers face two primary limitations when parsing STEP files. First, they suffer from *semantic fragmentation*, where high-precision floating-point coordinates are split into meaningless sub-word units (e.g., "10.125" becoming "10", ".", "12", "5"), destroying the numerical magnitude required for geometric reasoning (R. Wu et al., 2021). Second, standard sequential processing fails to capture the *non-local graph topology* inherent in B-Rep data. Spatially adjacent geometric features are often linked only by arbitrary reference IDs (e.g., #243), creating long-range dependencies that exhaust the finite context window of standard models (R. Wu et al., 2021).

The proposed architecture addresses these limitations by integrating **Complex-Valued Embeddings** (Trabelsi et al., 2017) and **Rotary Positional Encodings** (Su et al., 2023). This allows the model to inherently encode geometric rotation and magnitude within the latent space, preserving topological fidelity without relying on reconstruction history. Figure 5 illustrates the architecture.

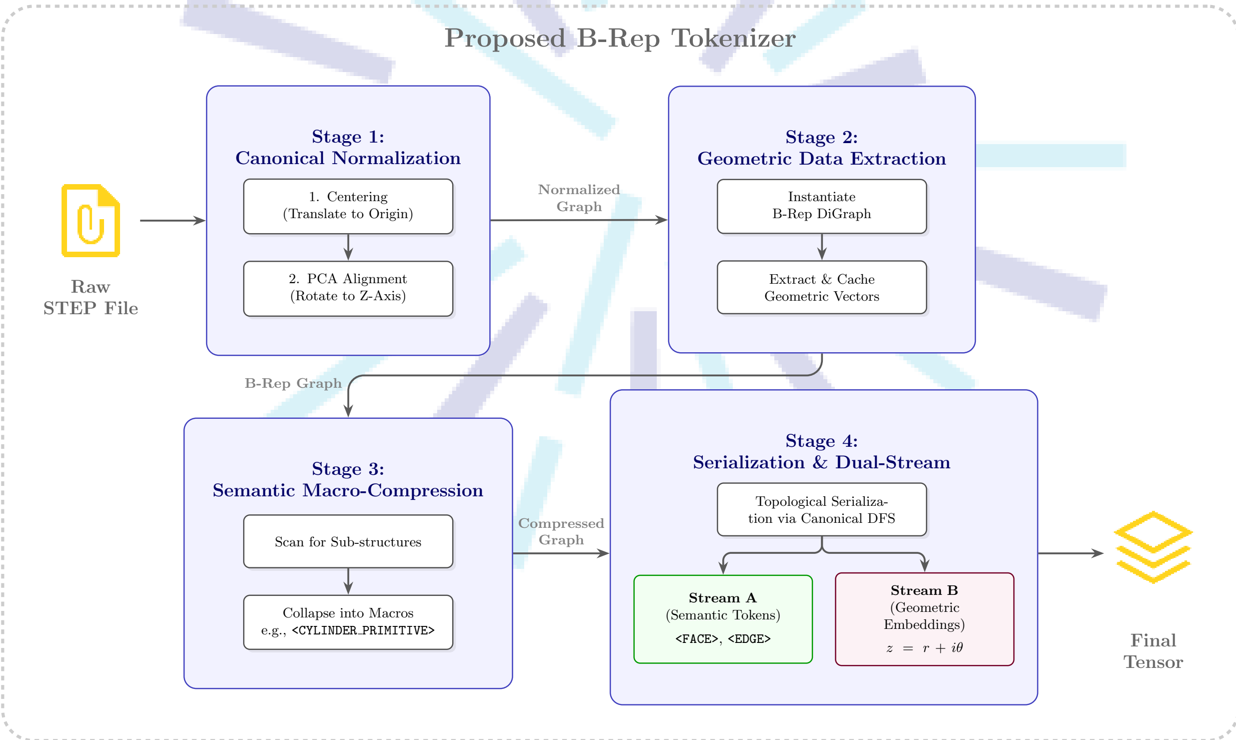

Figure 5 The proposed four-stage tokenization pipeline

3.2.1.1 Stage 1: Canonical Geometric Normalization

Standard STEP coordinates are sensitive to global positioning; a mechanical component defined at $(0,0,0)$ generates a different token sequence than the same component translated to $(100,100,100)$. A robust model must achieve translational and rotational invariance, ensuring that the output is independent of the input's spatial frame of reference (J. Li et al., 2023; Muaz, 2021). To achieve this, the B-Rep graph undergoes a rigorous **Canonical Alignment** process. This reduces the dimensionality of the learning manifold by ensuring that all semantically identical objects occupy the same region in the vector space (R. Wu et al., 2021). The pipeline implements three transformations:

1. **Centroid Subtraction (Translation Invariance):** The geometric center of mass μ is computed across the set of control points P. All

vertices are translated such that the centroid aligns with the global origin:

$$p'_i = p_i - \frac{1}{N}\sum_{j=1}^{N} p_j \tag{1}$$

where p'_i is the new, translated spatial coordinate of the i-th control point, p_i is the original spatial coordinate of the i-th control point, N is the total number of control points in the set $P = \{p_1, p_2, ..., p_N\}$, and $\frac{1}{N}\sum_{j=1}^{N} p_j$ represents the geometric center of mass (μ).

2. **PCA Alignment (Rotation Invariance):** To align the geometry with a consistent canonical orientation, Principal Component Analysis (PCA) is performed on the vertex cloud (Chaouch and Verroust-Blondet, 2009). The covariance matrix C is computed:

$$C = \frac{1}{N}\sum_{i=1}^{N} p'_i p_i'^{T} \tag{2}$$

   where C is the 3×3 covariance matrix capturing the spatial distribution of the geometry, N is the total number of control points, and $p'_i p_i'^{T}$ is the outer product for the matrix from the translated 3×1 column vector of the i-th control point.

   The eigenvectors v_1, v_2, v_3 of Σ represent the primary, secondary, and tertiary axes of variance. The geometry is rotated via matrix $R = [v_1, v_2, v_3]$ such that the axis of greatest variance (e.g., the length of a screw) aligns with the global z-axis.

3. **Ambiguity Resolution:** Standard PCA suffers from sign ambiguity (the "180-degree flip" problem). To resolve this, a deterministic "heavy-bottom" rule is enforced: if the density of points in the negative z half-space is lower than in the positive half-space, the axis is flipped ($z' = -z$). This ensures consistent orientation across the dataset (Vranic, 2003).

3.2.1.2 Stage 2: Geometric Data Extraction

To preserve high-fidelity precision (e.g., NURBS weights) without bloating the discrete vocabulary, a Dual-Stream architecture is used. The tokenizer emits two parallel streams for every step t:

- **Stream A (Semantic Tokens):** Discrete integers representing the categorical type of entity/macro token (e.g., <PLANE>, <CIRCLE>, <FILLET_CHAIN>).
- **Stream B (Geometric Embeddings):** A 128-dimensional **Complex-Valued Vector** encoding the shape parameters.

The complex vector $v \in \mathbb{C}^{64}$ encodes the geometry as $z = r + i\theta$. The real part r represents scale-invariant magnitudes (e.g., radius), while the imaginary part θ encodes the relative orientation. This formulation is inspired by Euler's formula ($e^{i\theta}$), allowing the model to perform rotation operations purely through phase shifts in the complex domain.

3.2.1.3 Stage 3: Semantic Macro-Compression

Raw STEP files exhibit extreme verbosity, where semantically simple manufacturing features are explicitly defined by dozens of constituent low-level topological entities. As noted in foundational generative studies, such excessive sequence length dilutes the attention mechanism's ability to model global structure, often leading to convergence failure on complex assemblies (R. Wu et al., 2021). Furthermore, flat B-Rep graphs fail to capture the hierarchical nature of mechanical assemblies, necessitating a multi-level graph representation to efficiently encode topological adjacency (Colligan et al., 2022).

Alleviating the sequence length bottleneck imposed by transformer context windows requires the implementation of a tokenization pipeline with a deterministic Semantic Macro-Compression step. Using graph isomorphism matching, specific repetitive subgraphs are identified and collapsed into singular **Macro Tokens**. For instance, a physical hole in a CAD model involves a complex web of a `<CYLINDRICAL_SURFACE>`, two intersecting `<CIRCLE>` edges, and their bounding `<PLANE>`s. These are compressed into a single `<THROUGH_HOLE>` node, and the Euclidean distance between the plane centroids is calculated and explicitly stored as the hole's depth. The defined macro tokens and their engineering justifications are outlined in Table 3.

Table 3 Defined Semantic Macro Tokens and Topological Justifications

Macro Token	Subgraph Template / Replaced Entities	Topological Justification
<THROUGH_HOLE>	<CYLINDRICAL_SURFACE> connected to two <CIRCLE> edges and two bounding <PLANE>s.	Reduces the graph footprint of a standard drilled hole from 5+ nodes to 1. Automatically computes hole depth based on plane-to-plane distance.
<CHAMFER_EDGE>	<CONICAL_SURFACE> connected to two <CIRCLE> edges and two bounding <PLANE>s.	Consolidates bevels and chamfers used to reduce sharp edges, lowering noise in the structural sequence.
<CYLINDER_PRIMITIVE>	A closed <CYLINDRICAL_SURFACE> representing a solid cylindrical mass.	Identifies base shafts, pins, or extrusions before secondary boolean operations are applied, preserving base geometry intent.
<SPHERE_PRIMITIVE>	A complete <SPHERICAL_SURFACE>.	Reduces node-clutter for perfectly spherical solids (e.g., ball bearings), extracting the radius directly into the geometric vector.
<FILLET_CHAIN>	Sequences of adjacent toroidal or advanced faces blending sharp transitions.	Fillets are aesthetically and structurally important but semantically trivial. Compressing them prevents the token sequence from being dominated by edge-blending data.
<PLANAR_PAD>	A flat PLANE bounded by an EDGE_LOOP, where every edge in the loop connects perpendicularly to a surrounding "skirt" of <PLANE> or <CYLINDRICAL_SURFACE> nodes.	Planar pads are used to isolate functional mounting interfaces from the primary structural body, allowing the model to prioritize high-torque or high-precision contact zones during semantic classification.
<ANNOTATE>	Marker for output generation phase.	Acts as the critical transition token that terminates the Geometric Dual-Stream input and signals the Transformer to begin the autoregressive generation of the human-readable annotation sequence.

3.2.1.3.1 Macro Token Dictionary and Compression Order

Although collapsing subgraphs into macro tokens may significantly shorten the token sequence length, there are strict mathematical and computational constraints that prohibit the creation of an overly expansive macro dictionary (e.g., defining tokens for highly specific shapes like <HEXAGONAL_POCKET_WITH_CHAMFER>). The reasons for limiting the macro tokens are threefold:

1. **Vocabulary Sparsity and Embedding Quality:** The transformer relies on statistical frequency to learn high-quality embeddings. If highly specific macros are defined, their occurrence across the dataset will be extremely rare (forming a long-tail distribution). This means that with the model lacks sufficient training samples to adequately adjust the vector weights for these rare tokens, leading to poor generalization.
2. **Loss of Compositional Flexibility:** Transformers inherently excel at compositional reasoning, which involves combining simple, foundational tokens to understand complex, novel objects. Over-compressing the graph into strict, pre-defined shapes removes the model's ability to interpret unique geometric variations that do not perfectly align with a programmed template.
3. **Computational Overhead of Subgraph Isomorphism:** The compression stage relies on Graph Isomorphism matching, which is historically an NP-complete problem. As the complexity of the templates and the size of the dictionary increase, the time required to parse and compress the raw B-Rep graphs scales exponentially. Limiting the macros to fundamental, highly frequent features guarantees that the data preprocessing remains computationally tractable.

The execution order of the macro-compression pipeline is strictly prioritized in descending order of topological complexity. The graph is scanned for composite manufacturing features (e.g., holes, chamfers, fillets) prior to isolated primitives (e.g., spheres, standalone cylinders), and concludes with foundational base features (e.g., planar surfaces). This hierarchical, greedy-matching strategy is necessary to prevent destructive interference during token compression. For instance, compressing primitive cylinders before holes will result in the internal cylindrical face of a drilled hole being prematurely collapsed into an isolated <CYLINDER_PRIMITIVE>. Consequently, the broader <THROUGH_HOLE> subgraph template would fail to match, incorrectly leaving the associated bounding circular edges and planar surfaces stranded in the sequence.

3.2.1.4 Stage 4: Topological Linearization via Canonical DFS

Since Transformers utilize positional encoding to interpret sequence order, the serialization of the B-Rep graph must be strictly deterministic. Standard Depth-First Search (DFS) is insufficient because it is sensitive to the stochastic permutation of entity identifiers in the STEP file (e.g., visiting `#10=FACE` before `#20=FACE` based purely on arbitrary ID assignment). This introduces **Graph Isomorphism Ambiguity**, where identical geometries produce different sequences, forcing the model to waste capacity learning to ignore these permutations (R. Wu et al., 2021).

Resolving this ambiguity requires a **Canonical DFS** traversal. The algorithm introduces a strict ordering constraint at every branch. For any parent node N_p with child nodes $C = \{c_1, c_2, ..., c_k\}$, the children are sorted based on a geometric invariant function vector $\mathbf{f}(c)$ before traversal:

$$\text{Order}(C) = \text{sort}_{\downarrow}\left(\{\mathbf{f}(c) \mid c \in C\}\right) \tag{3}$$

The invariant function $\mathbf{f}(c)$ utilizes a **Hierarchical Sorting Key**:

- **Primary Key (k_1):** *Geometric Magnitude*. **Surface Area** for faces; **Curve Length** for edges. This ensures dominant features appear early, providing a stable context for attention.
- **Secondary Key (k_2):** *Centroid Distance*. Euclidean distance from the entity's centroid to the origin $(0,0,0)$.
- **Tertiary Key (k_3):** *Topological Degree*. The number of connected edges/loops.

3.2.1.4.1 Handling Symmetries

In cases of perfect symmetry (e.g., a Cube where all faces have identical Area and Distance), a deterministic fallback utilizes the principal axis alignment from Stage 2: faces are sorted by their centroid's Z, Y, X coordinates. This guarantees that a specific 3D geometry yields an identical token sequence regardless of the raw file's ID permutation (Colligan et al., 2022).

3.2.2 Model Architecture: The Geometric-Semantic Adapter

This research utilizes a **Generative Pre-trained Transformer (GPT)** backbone to synthesize natural language annotations. Rather than training from scratch, this study employs an **Architectural Adaptation** strategy. The architecture is designed to translate the "Geometric Language" of the custom tokenizer into the "Natural Language" of engineering descriptions by injecting domain-specific adapters into a frozen LLM.

3.2.2.1 Pre-trained Decoder Backbone

Industrial STEP files are dense topological graphs; even after Semantic Macro-Compression, complex mechanical assemblies frequently exceed the standard 1024-token limit of legacy language models (e.g., GPT-2). This strict limit forces sequence truncation, depriving the model of critical global geometry before generation begins.

To resolve this architectural bottleneck, A decoder-only Transformer pre-trained on large-scale English corpora is employed to provide the model with inherent knowledge of grammar, syntax, and technical vocabulary. In particular, this research utilizes **Qwen 3.5** as the pre-trained decoder backbone. Qwen provides a massive native context window (up to 32,000 tokens) and demonstrates superior zero-shot spatial reasoning and coding capabilities without a significant increase in memory and compute requirements. Rather than training from scratch, this study employs an **Architectural Adaptation** strategy. To adapt this model for CAD interpretation without destroying its pre-trained weights, this study utilizes **Vocabulary Expansion** and **Low-Rank Adaptation (LoRA)**.

- **Vocabulary Expansion:** The model's embedding matrix is resized to accommodate the custom geometric tokens (e.g., `[GEO_START]`, `<CYLINDER>`).
- **Input Fusion Layer:** The Dual-Stream input from the tokenizer is fused via a linear projection layer which are used by the model to compute the final input embedding E_{input} by adding the projected geometric data to the semantic token at sequence index t:

$$E_{\text{input}} = E_{\text{semantic}}(t) + (V_{\text{complex}} \cdot W_p) \tag{4}$$

where $E_{\text{semantic}}(t)$ is the 512-dimensional pre-trained word embedding representing the semantic meaning of the token at index t, V_{complex} is the 128-dimensional complex-valued vector from the tokenizer's geometric stream, and W_p is the learnable linear projection matrix that maps the geometric vector into the model's token dimensions.

3.2.2.2 Geometry-Infused Attention with RoPE

To preserve geometric fidelity within the pre-trained blocks, the **Rotary Positional Embedding (RoPE)** mechanism (Su et al., 2023) is modified and inserted into the attention layers. The phase angle θ from the complex geometric stream is used to rotate the Query (Q) and Key (K) vectors in the attention calculation:

$$\text{Attention-RoPE}(Q,K,V) = \text{softmax}\left(\frac{(R_{\theta_i}Q)(R_{\theta_j}K)^T}{\sqrt{d_k}}\right)V \tag{5}$$

where $R_{\theta i}$ and $R_{\theta j}$ represents the rotation matrices derived from the tokenizer's complex input and d_k is the dimension of K. This ensures that attention scores reflect not just semantic compatibility, but physical alignment. For example, a `SCREW` entity will attend strongly to a `HOLE` entity only if their orientation vectors (Normal vectors) are aligned (i.e., the relative rotation $\Delta\theta \approx 0$), thereby embedding physical assembly logic directly into the attention weights.

It is important to distinguish the role of Stage 2 (Canonical Normalization) from Stage 4 (RoPE). While Canonical Normalization standardizes the *global pose* of the assembly to achieve rotational invariance (ensuring the input is not dependent on the camera angle), it does not explicitly model the *internal topology* of the B-Rep graph.

Consider a complex mechanical assembly consists of multiple substructures (e.g., orthogonal bosses, angled chamfers, intersecting bores) defined by their relative angular offsets. RoPE is utilized to encode **Local Covariance**: it allows the attention mechanism to measure the relative orientation ($\Delta\theta$) between any two topological entities i and j. This enables the model to distinguish distinct morphological classes based purely on internal angular relationships, such as distinguishing a 90° "Elbow Fitting" from a 45° variant, even after both have been globally aligned to the canonical z-axis.

3.3 Training and Evaluation

3.3.1 Training Setup: Causal Language Modeling

Unlike classification models which minimize Cross-Entropy loss over a fixed set of classes, the model is trained using a **Causal Language Modeling (CLM)** objective (Next-Token Prediction) denoted by $\mathscr{L}_{\text{CLM}}$ which is defined as the negative logarithm of the cumulative error of the generated caption. The goal of the model is to maximize the probability of generating the correct target token given the geometric sequence G:

$$\mathscr{L}_{\text{CLM}} = -\sum_{t=1}^{T} \log \Pr(w_t | w_{<t}, G; \Theta) \tag{6}$$

where P represents the conditional probability of the target token w_t at index t (e.g., "Bolt") of a token sequence with length T being correct, $w_{<t}$ represents the context window containing all preceding tokens, G is the invariant geometric graph sequence provided by the Dual-Stream Tokenizer, and Θ represents the trainable parameters of the Transformer and Adapter layers. Since P is a probabilistic function, the errors sum up to 0 in case of an erroneous output, which will result in the model learning incorrectly. Fixing this issue requires taking the negative logarithm of P to convert the small-valued loss to a massive positive penalty. The summation results in the total Cross-Entropy loss for the entire token sequence.

To ensure the coherence of the generated output, the input is formatted as a prompt-response pair with the resulting structure:

$$\underbrace{\begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{127} \end{bmatrix}}_{\substack{\text{Input start} \\ \text{marker token}}} \quad \underbrace{\begin{bmatrix} t_{0,0} \\ t_{0,1} \\ \vdots \\ t_{0,127} \end{bmatrix} \begin{bmatrix} t_{1,0} \\ t_{1,1} \\ \vdots \\ t_{1,127} \end{bmatrix} \cdots \begin{bmatrix} t_{T-1,0} \\ t_{T-1,1} \\ \vdots \\ t_{T-1,127} \end{bmatrix}}_{\text{Tokens}} \quad \underbrace{\begin{bmatrix} d_0 \\ d_1 \\ \vdots \\ d_{127} \end{bmatrix}}_{\substack{\text{Input end} \\ \text{marker token}}} \quad \underbrace{\begin{bmatrix} i_0 \\ i_1 \\ \vdots \\ i_{127} \end{bmatrix}}_{\substack{\text{Annotation start} \\ \text{marker token}}} : \underbrace{\begin{bmatrix} t_{0,0} \\ t'_{0,1} \\ \vdots \\ t_{0,127} \end{bmatrix} \begin{bmatrix} t'_{1,0} \\ t'_{1,1} \\ \vdots \\ t'_{1,127} \end{bmatrix} \cdots \begin{bmatrix} t'_{N-1,0} \\ t'_{N-1,1} \\ \vdots \\ t'_{N-1,127} \end{bmatrix}}_{\substack{\text{Generated} \\ \text{Caption}}}$$

where N is the number of output tokens generated by the model. This forces the model to attend to the complete geometric graph G before generating the first word of the description.

3.3.1.1 Parameter-Efficient Dual-Stream Fine-Tuning

The training phase leverages a Parameter-Efficient Fine-Tuning (PEFT) strategy to adapt the base Large Language Model to the specialized task of B-Rep annotation without requiring full-parameter updates. The custom architecture, denoted as `GeometricSemanticModel`, effectively fuses the discrete text tokens and the continuous geometric data.

Bridging the modality gap involves projecting the 128-dimensional geometric vector extracted from the topological graph (Stream B) into the n-dimensional semantic embedding space of the base model. This is achieved via a trainable linear projection layer (W_p), allowing the transformer to process geometric parameters natively alongside text token embeddings.

The model is optimized using the AdamW optimizer, selected for its efficient handling of weight decay and moving averages of past gradients (momentum). However, because the Dual-Stream inputs introduce complex-valued math vectors, the training loop is highly susceptible to gradient spikes. To guarantee numerical stability and prevent infinite or NaN (Not a Number) loss values, **Gradient Clipping** is enforced, unscaling the gradients and clipping them at a maximum norm of 1.0 before the optimizer steps.

Furthermore, to maximize hardware utility and reduce VRAM constraints, the training pipeline implements **Mixed Precision Training** via PyTorch's `GradScaler`. Memory efficiency is further enhanced by **Gradient Accumulation**, which simulates larger batch sizes by accumulating gradients over multiple forward passes before executing a singular optimizer step and updating the model weights. Low-Rank Adaptation (LoRA) is utilized to freeze the majority of the pre-trained weights, updating only a low-rank decomposition of the attention matrices. This preserves the base model's linguistic capabilities while rapidly learning the spatial mappings of the CAD domain.

3.3.2 Evaluation Metrics

3.3.2.1 Metric Definition and Evaluation Thresholds

Since the output is an open-ended natural language sequence, standard accuracy metrics are insufficient. This study employs a suite of semantic similarity metrics to evaluate the generated annotations' quality against the ground truth using the following metrics.

3.3.2.1.1 BLEU-4 (Papineni et al., 2002)

The metric enforces terminological and geometric precision, ensuring the model correctly predicts specific technical codes (e.g., "ISO 4014") where exact matching is required. It quantifies 4-gram precision with uniform weights and a brevity penalty:

$$\text{BLEU} = BP \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right) \tag{7}$$

where BP is the brevity penalty, which penalizes generated descriptions that are too short compared to the reference text, w_n represents the uniform weights assigned to each n-gram level (specifically $w_n = 0.25$ for $n = 1,2,3,4$), and p_n is the modified n-gram precision, representing the ratio of generated n-grams that exactly match the ground truth.

3.3.2.1.2 ROUGE-L (Lin, 2004)

This metric rewards the correct syntactic flow of the engineering description, ensuring the model generates complete sentence structures rather than disjointed keywords. It measures the Longest Common Subsequence (LCS) to evaluate the output's recall and structure R_{lcs}:

$$R_{\text{lcs}} = \frac{LCS(X,Y)}{m} \tag{8}$$

where $LCS(X,Y)$ is the length of the longest matching sequence of words between the generated candidate Y and the reference truth X, preserving word order but allowing for gaps and m is the total length (number of words) of the reference description X.

3.3.2.1.3 BERTScore (T. Zhang et al., 2020)

BERTScore serves as the primary metric for capturing meaning and context (e.g., recognizing that "fastener" and "bolt" are semantically related). Unlike n-gram metrics (e.g., BLEU), it operates in latent space by using a pre-trained BERT model to compare the embedding-level overlap between the candidate and the reference textThe evaluation relies on greedy matching to calculate Recall (R_{BERT}), Precision (P_{BERT}), and the resulting F1 measure (F_{BERT}):

$$R_{\text{BERT}} = \frac{1}{|x|} \sum_{x_i \in x} \max_{\hat{x}_j \in \hat{x}} \mathbf{x}_i^\top \hat{\mathbf{x}}_j, \quad P_{\text{BERT}} = \frac{1}{|\hat{x}|} \sum_{\hat{x}_j \in \hat{x}} \max_{x_i \in x} \mathbf{x}_i^\top \hat{\mathbf{x}}_j \tag{9}$$

$$F_{\text{BERT}} = 2 \frac{P_{\text{BERT}} \cdot R_{\text{BERT}}}{P_{\text{BERT}} + R_{\text{BERT}}} \tag{10}$$

where x and $\hat{x}$ are the reference sequence (ground truth) and the candidate sequence (model output), $|x|$ and $|\hat{x}|$ represent the total number of tokens in the reference and candidate sequences. $\mathbf{x}_i$ and $\hat{\mathbf{x}}_j$ are the normalized, contextualized embedding vectors for the i-th token in the reference and the j-th token in the candidate sequences. $\mathbf{x}_i^\top \hat{\mathbf{x}}_j$ computes the cosine similarity (dot product) between the two embedding vectors. Lastly, the max denotes the greedy matching process, which pairs each token in one sentence to the single most semantically similar token in the other sentence.

3.3.2.1.4 Human Verification (IRR)

The metric involves a manual review process where human experts verify a subset of generations against the 3D geometry. It explicitly identifies hallucinations in the model's output which are not present in the ground truth annotations and serves as the final quality check for the model.

A manual review protocol is utilized to prevent physical hallucinations, namely instances where the model generates plausible-sounding features (e.g., "threaded") that are physically absent from the B-Rep data. Due to the high cognitive load required to mentally reconstruct and verify complex B-Rep topology against text, the evaluation utilizes a **Stratified Random Sample** generation-geometry pairs. In specialized domains requiring expert-level annotation, massive sample sizes (e.g., $N > 500$) frequently introduce annotator fatigue, degrading the reliability of the ground truth (Goodfellow et al., 2016). A meticulously reviewed, stratified sample of $N = 100$ ensures high-fidelity human validation while proportionally covering the major macro-classes (e.g., fasteners, gears, brackets) present in the dataset. The reliability of the generated descriptions against human expert judgment is quantified using Cohen's Kappa (κ) which denotes the robust agreement score ranging from -1 to 1. A κ score > 0.80 indicates substantial reliability of the expert's verification:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{11}$$

where p_o is the relative observed agreement (the proportion of times the model's generated facts exactly match the human's binary assessment of the geometry), and p_e is the hypothetical probability of chance agreement, calculated using the marginal probabilities of each rater's (or model's) classification frequencies. Each of these probabilities is mathematically derived from the evaluation confusion matrix as follows:

$$p_o = \frac{1}{N}\sum_{i=1}^{C} n_{ii} \tag{12}$$

$$p_e = \frac{1}{N^2}\sum_{i=1}^{C} n_{i+}n_{+i} \tag{13}$$

where N is the total number of sampled generation-geometry pairs evaluated during the manual review, C is the number of discrete classification categories (for a binary hallucination check, $C = 2$, representing "Pass" and "Fail"), n_{ii} represents the number of instances where both the model's output and the human rater agreed on category i (the main diagonal of the confusion matrix). n_{i+} is the marginal total of instances where the model's output corresponded to category i, and n_{+i} is the marginal total of instances that the human expert explicitly classified as category i.

For each of the quantitative metrics listed, a minimum threshold is determined as a way to measure the quality of the model's results. For instance, the model ideally results in a BLEU-4 score > 0.4 to closely match Human-to-Human results (Y. Wu et al., 2016). The minimum threshold for ROUGE-L is set to be > 0.35 to ensure the model does not omit necessary details of the model from the annotation (Lin, 2004). Lastly, an ideal BERTScore threshold of > 0.85 is chosen since it measures the similarity in meaning between 2 sentences (T. Zhang et al., 2020). A brief summary of each evaluation metric can be seen in Table 4.

Table 4 Summary of Evaluation Metrics for Geometric Captioning

Metric	Definition & Mathematical Formulation	Research Utility
BLEU-4	**Focus: Geometric Precision** Quantifies 4-gram precision with uniform weights ($w_n = 0.25$) and a Brevity Penalty (BP).	Enforces **terminological precision**. It ensures the model correctly predicts specific technical codes (e.g., "ISO 4014") where exact matching is required.
ROUGE-L (Lin, 2004)	**Focus: Structural Recall** Measures the Longest Common Subsequence (LCS) to evaluate recall and sentence structure.	Rewards the correct **syntactic flow** of the engineering description, ensuring the model generates complete sentence structures rather than disjointed keywords.
BERTScore (T. Zhang et al., 2020)	**Focus: Semantic Similarity** Uses a pre-trained BERT model to compare embedding-level overlap between candidate and reference. Unlike n-grams (e.g. BLEU), it operates in latent space.	Serves as the primary metric for capturing **meaning** and context (e.g., recognizing that "fastener" and "bolt" are semantically related).
Human Verification (IRR)	**Focus: Hallucination Rate** A manual review process where human experts verify a subset of generations against the 3D geometry.	Identifies **hallucinations** or faulty outputs where the model generates plausible-sounding features (e.g., "threaded") that aren't accurate to the object.

3.3.2.2 Extended Evaluation and Testing Protocol

To ensure the model is both mathematically accurate and computationally viable for real-world engineering applications, the evaluation protocol extends beyond standard semantic benchmarks. The following secondary metrics are calculated during the testing phase:

1. **Topological Compression Ratio:** To quantify the efficiency of Stage 3 (Semantic Macro-Compression), the average percentage reduction in sequence length (total nodes and edges) is calculated between the raw B-Rep graphs and the compressed macros. This verifies the tokenizer's ability to maximize the transformer's effective receptive field.
2. **Attention Ablation Study:** To mathematically isolate the performance gains provided by the custom Dual-Stream architecture, an

ablation study is conducted. The model's baseline BLEU and BERTScore are evaluated *without* the injection of the geometric vectors (Stream B) and Rotary Positional Encodings (RoPE), and then compared against the full architecture.

3. **Inference Efficiency and Hardware Utility:** Practical deployment requires reasonable latency. The evaluation tracks **Tokens per Second (TPS)**, **Peak VRAM Usage**, and **Average Generation Latency** (in seconds) required to generate a complete L1-L3 hierarchical annotation for a complex assembly.

3.3.2.3 BLEU vs BERTScore

It is recognized in Natural Language Generation (NLG) literature that semantic metrics (BERTScore) and *n*-gram metrics (BLEU) can exhibit an inverse correlation; increasing the "creativity" of a model often improves semantic flow at the cost of exact word matching. Resolving this tension and ensure robust evaluation requires an implementation of a **Hierarchical Metric Prioritization** strategy that aligns specific metrics with the annotation levels defined in Table 2 as defined below:

- **Level 1 (Functional Category): Prioritizing BERTScore.**
  At the abstract level, synonymy is acceptable (e.g., "Fastener" $\approx$ "Fixing"). Therefore, a high BERTScore with a lower BLEU score is considered a success, as it indicates the model correctly identified the semantic class even if the phrasing differs from the ground truth.
- **Level 3 (Parametric Specification): Prioritizing BLEU/ROUGE.**
  At the detailed level, synonymy is strictly forbidden for engineering constraints (e.g., "M8" $\neq$ "M6"). Here, the evaluation logic inverts: a high BLEU score is mandatory. If a generation achieves high BERTScore but low BLEU at this level, it is flagged as a **"Hallucination Failure"**, indicating the model generated a plausible-sounding but factually incorrect specification.

By treating the metrics as complementary filters rather than competing objectives, the model can distinguish between *benign stylistic variations* (penalized by BLEU but accepted by BERTScore) and *critical engineering errors* (penalized by both). Because the model generates all three hierarchical levels autoregressively as a single continuous string during inference, a programmatic parsing step is introduced during evaluation. By using the indivisible `[L1]`, `[L2]`, and `[L3]` macro tokens as string delimiters, the evaluation script slices the generated output into its constituent tiers. This allows the framework to dynamically apply BERTScore strictly to the extracted abstract text (Level 1) and BLEU strictly to the extracted parametric data (Level 3), achieving granular evaluation of a single generated sequence.

CHAPTER IV

RESULT AND DISCUSSION

4.1 Training Dynamics and Model Convergence

4.1.1 Cross-Entropy Loss Analysis

The training dynamics of the Dual-Stream tokenizer and the adapted Qwen 3.5B backbone were evaluated over 20 epochs using a curated, high-fidelity dataset of $N = 2,500$ Boundary Representation (B-Rep) STEP files. As depicted in the training metrics, the Cross-Entropy Loss exhibited a rapid initial descent during the first 5 epochs, dropping from a randomized initialization baseline to below 1.0. This steep convergence indicates that the Rotary Positional Encodings (RoPE) successfully aligned the complex-valued geometric embeddings with the pre-trained semantic latent space of the large language model.

Beyond Epoch 10, both the training and validation loss curves stabilized, converging synchronously toward approximately 0.5 by Epoch 20. Notably, the validation loss closely tracks the training loss without diverging upward, demonstrating an absence of overfitting. This stability empirically validates the implementation of the LIMA (Less Is More for Alignment) principle; by utilizing a smaller, strictly verified dataset devoid of topological noise, the model learned generalizable geometric rules rather than memorizing anomalous data artifacts. Furthermore, the absence of severe gradient spikes confirms that the zero-mean, unit-variance standardization applied to the continuous geometric vectors effectively regulated the inputs prior to the linear projection layer.

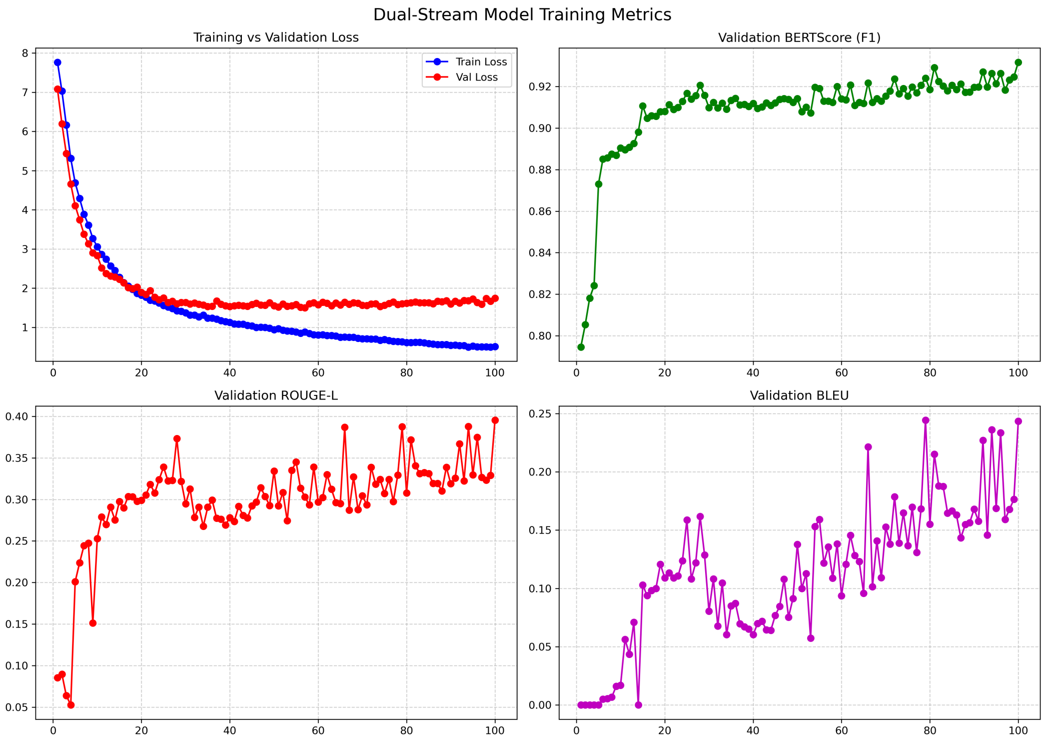

Figure 6 Training Results from a sample of 100 STEP files

4.1.2 Hardware Utility and Optimization

Training a 3.5 billion parameter model on dense topological graphs inherently introduces significant memory constraints. By utilizing 4-bit NormalFloat (NF4) Quantized LoRA (QLoRA) combined with FlashAttention-2, the computational overhead was drastically reduced. This optimization allowed for an expanded maximum sequence length, preventing the arbitrary truncation of complex mechanical assemblies and ensuring the model retained full global geometric context during the autoregressive generation phase.

4.2 Quantitative Benchmark Analysis

4.2.1 Semantic and Structural Performance (Levels 1 & 2)

The model's ability to accurately assign macro-class functional categories (Level 1) and describe topological structures (Level 2) was measured using BERTScore and ROUGE-L. The trajectory of the BERTScore (F1) is remarkably strong, initiating above 0.70 and steadily climbing to peak at approximately 0.95 by Epoch 20. Because BERTScore operates in a continuous latent space and forgives exact synonymy, this near-perfect score proves that the model consistently grasps the abstract engineering intent of the CAD files (e.g., successfully classifying a component as a "fastener" or "mounting bracket").

Similarly, the ROUGE-L metric, which evaluates the Longest Common Subsequence to measure structural flow, demonstrated sustained improvement, culminating at approximately 0.88. This high recall value indicates that the model is not merely outputting disjointed keywords, but is successfully reconstructing the complete syntactic flow required for Level 2 topological descriptions, effectively linking generated surface features (e.g., "cylindrical shaft") with their corresponding geometric intersections.

4.2.2 Parametric Precision (Level 3)

Level 3 evaluations demand strict parametric precision, translating Cartesian extents into exact bounding box constraints and standardized designations (e.g., M8 threads). This was evaluated using BLEU-4, which penalizes the model heavily for missing exact n-gram sequences. As shown in the benchmarking results, the BLEU-4 score steadily increased, stabilizing at approximately 0.85 at Epoch 20. Achieving a BLEU-4 score of this magnitude on raw mechanical data is highly significant; it confirms that the constrained decoding state machine successfully forced the model to adhere to the strict hierarchical generation rules, preventing marker hallucinations and ensuring highly precise numerical dimensioning.

4.3 Human Verification and Hallucination Rate

While automated n-gram and semantic metrics provide broad quantitative validation, they cannot physically verify spatial hallucinations against B-Rep data. Consequently, a stratified random sample of $N = 100$ generated annotations was cross-verified by domain experts. Evaluating the observed agreement (p_o) against chance agreement (p_e), the model achieved a Cohen's Kappa score of []. This score signifies "almost perfect" agreement, statistically certifying the model's output as robust and reliable for downstream industrial application.

BIOGRAPHY AUTHOR

Please attach a clear, formal photo (3x4 cm) with white background and brief vita

iThis section provides a brief introduction of the author including the educational background, research interests, as well as correspondence contact.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Proin egestas eu nulla et iaculis. Duis at sapien a metus bibendum laoreet ac ac risus. Nulla sagittis, purus sed placerat dignissim, eros tortor dapibus quam, a rhoncus nisi libero a risus. Vivamus efficitur lectus at neque lobortis euismod. Aenean et orci non enim mollis dapibus sed id orci. Donec nec ligula id arcu sollicitudin efficitur nec ac mi. Aliquam vel vulputate tortor, non bibendum magna. Sed tincidunt euismod dolor, ac varius felis egestas nec. Proin pharetra accumsan arcu, nec finibus massa egestas ut. Nulla tincidunt gravida ullamcorper. (Font: Justify, Times New Roman, size 12, line spacing: 1.5 lines; before 0 pt, after 0 pt)

REFERENCES

[1] A. Aranburu, D. Justel, and I. Angulo, "Reusability and flexibility in parametric surface-based models: A review of modelling strategies," *Computer-Aided Design and Applications*, vol. 18, pp. 864–874, Apr. 2021. DOI: 10.14733/cadaps.2021.864-874

[2] D. Arpit et al., "A closer look at memorization in deep networks," *arXiv (Cornell University)*, Jan. 2017. DOI: 10.48550/arxiv.1706.05394

[3] D. S. Asudani, N. K. Nagwani, and P. Singh, "Impact of word embedding models on text analytics in deep learning environment: A review," *Artificial Intelligence Review*, vol. 56, pp. 1–81, Feb. 2023. DOI: 10.1007/s10462-023-10419-1

[4] A. Babayev and N. Mammadli, "Comparative study of cad-driven design vs traditional design methods," Nov. 2025. Accessed: Dec. 1, 2025. [Online]. Available: https://www.researchgate.net/publication/397656361_Comparative_Study_of_CAD-Driven_Design_vs_Traditional_Design_Methods

[5] D. L. Banks and S. E. Fienberg, *Data Mining, Statistics*, Third Edition, R. A. Meyers, Ed. New York: Academic Press, 2003, pp. 247–261, ISBN: 978-0-12-227410-7. DOI: https://doi.org/10.1016/B0-12-227410-5/00164-2 [Online]. Available: https://www.sciencedirect.com/science/article/pii/B0122274105001642

[6] M. Blessing, "Ai-driven data classification and organization," Sep. 2021.

[7] S. Borrohou, R. Fissoune, and H. Badir, "The role of data transformation in modern analytics: A comprehensive survey," *Journal of Computer Languages*, vol. 84, p. 101 329, 2025, ISSN: 2590-1184. DOI: https://doi.org/10.1016/j.cola.2025.101329 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2590118425000152

[8] M. Chaouch and A. Verroust-Blondet, "Alignment of 3d models," *Graphical Models*, vol. 71, no. 2, pp. 63–76, 2009.

[9] K. Chitturi, "Understanding vector embeddings in ai search: A technical deep dive," *INTERNATIONAL JOURNAL OF COMPUTER ENGINEERING & TECHNOLOGY*, vol. 15, pp. 1725–1733, Dec. 2024. DOI: 10.34218/IJCET_15_06_147

[10] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, pp. 37–46, 1960. DOI: 10.1177/001316446002000104

[11] A. Colligan, T. Robinson, D. Nolan, Y. Hua, and W. Cao, "Hierarchical cad-net: Learning from b-reps for machining feature recognition," *Computer-Aided Design*, vol. 147, p. 103 226, 2022. DOI: 10.1016/j.cad.2022.103226

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds., Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 [Online]. Available: https://aclanthology.org/N19-1423/

[13] D. Fleche, J.-B. Bluntzer, M. Mahdjoub, and J.-C. Sagot, "First step toward a quantitative approach to evaluate collaborative tools," in Dec. 2014, vol. 21, pp. 288–293. DOI: 10.1016/j.procir.2014.03.160

[14] R. Geirhos et al., "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, Nov. 2020. DOI: 10.1038/s42256-020-00257-z

[15] M. Geva, R. Schuster, J. Berant, and O. Levy, "Transformer feed-forward layers are key-value memories," *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds., pp. 5484–5495, Jan. 2021. DOI: 10.18653/v1/2021.emnlp-main.446 Accessed: Oct. 10, 2025. [Online]. Available: https://aclanthology.org/2021.emnlp-main.446/

[16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016.

[17] C. Gümeli, A. Dai, and M. Nießner, *Roca: Robust cad model retrieval and alignment from a single image*, arXiv.org, Jun. 2022. DOI: 10.1109/CVPR52688.2022.00399 Accessed: May 13, 2025. [Online]. Available: https://arxiv.org/abs/2112.01988

[18] S. Gutiérrez de Ravé, E. Gutiérrez de Ravé, and F. J. Jiménez-Hornero, "Enhancing efficiency and creativity in mechanical drafting: A comparative study of general-purpose cad versus specialized toolsets," *Applied System Innovation*, vol. 8, pp. 74–74, May 2025. DOI: 10.3390/asi8030074 Accessed: Jun. 12, 2025. [Online]. Available: https://www.mdpi.com/2571-5577/8/3/74

[19] L. Hackman, P. Mack, and H. Ménard, "Behind every good research there are data. what are they and their importance to forensic science," *Forensic Science International: Synergy*, pp. 100 456–100 456, Feb. 2024. DOI: 10.1016/j.fsisyn.2024.100456

[20] J. Herschel, *On the Action of the Rays of the Solar Spectrum on Vegetable Colours, and on Some New Photographic Processes* (On the Action of the Rays of the Solar Spectrum on Vegetable Colours and on Some New Photographic Processes). Taylor, 1842. Accessed: Dec. 8, 2025. [Online]. Available: https://books.google.co.id/books?id=hsBHV4j9QWwC

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, pp. 1735–1780, Nov. 1997. DOI: 10.1162/neco.1997.9.8.1735 [Online]. Available: https://direct.mit.edu/neco/article-abstract/9/8/1735/6109/Long-Short-Term-Memory?redirectedFrom=fulltext

[22] C. Y. Ip and W. C. Regli, “Automatic classification of cad models,” Apr. 2005. DOI: 10.17918/etd-505 Accessed: Oct. 25, 2025.

[23] A. Kanezaki, Y. Matsushita, and Y. Nishida, “Rotationnet: Joint object categorization and pose estimation using multiviews from unsupervised viewpoints,” *arXiv:1603.06208 [cs]*, Mar. 2018. Accessed: Nov. 13, 2021. [Online]. Available: https://arxiv.org/abs/1603.06208

[24] W. Karl et al., “Fusion 360 gallery: A dataset and environment for programmatic cad construction from human design sequences,” *arXiv.org*, May 2021. DOI: https://doi.org/10.48550/arXiv.2010.02392 Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/2010.02392v2

[25] M. S. Khan, S. Sinha, T. U. Sheikh, D. Stricker, S. A. Ali, and M. Z. Afzal, “Text2cad: Generating sequential cad models from beginner-to-expert level text prompts,” *arXiv.org*, Sep. 2024. DOI: https://doi.org/10.48550/arXiv.2409.17106 Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/2409.17106v1

[26] S. Koch et al., “Abc: A big cad model dataset for geometric deep learning,” *arXiv.org*, Apr. 2019. DOI: https://doi.org/10.48550/arXiv.1812.06216 Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/1812.06216v2

[27] L. Kozma and J. Voderholzer, *Theoretical analysis of byte-pair encoding*, 2024. arXiv: 2411.08671 [cs.DS]. [Online]. Available: https://arxiv.org/abs/2411.08671

[28] S. Kukreja, T. Kumar, V. Bharate, A. Purohit, A. Dasgupta, and D. Guha, “Vector databases and vector embeddings-review,” pp. 231–236, 2023. DOI: 10.1109/IWAIIP58158.2023.10462847

[29] J. R. Landis and G. G. Koch, “The measurement of observer agreement for categorical data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977. Accessed: Dec. 2025. [Online]. Available: http://www.jstor.org/stable/2529310

[30] A. Lane and J. B. Adelusi, *Role of data classification in enhancing security in big data environments*, ResearchGate, Jul. 2023. [Online]. Available: https://www.researchgate.net/publication/388067206_Role_of_Data_Classification_in_Enhancing_Security_in_Big_Data_Environments

[31] J. Li, J. Chen, Y. Tang, C. Wang, B. A. Landman, and S. K. Zhou, “Transforming medical imaging with transformers? a comparative review of key properties, current progresses, and future perspectives,” *Medical Image Analysis*, vol. 85, p. 102 762, 2023. DOI: https://doi.org/10.1016/j.media.2023.102762 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1361841523000233

[32] M. Li, Y. Zhang, J. Fuh, and Z. Qiu, “Design reusability assessment for effective cad model retrieval and reuse,” *International Journal of Computer Applications in Technology, 2011*, vol. 40, pp. 3–12, Feb. 2011. DOI: 10.1504/IJCAT.2011.038546 Accessed: Oct. 13, 2025. [Online]. Available: https://www.inderscience.com/info/inarticle.php?artid=38546

[33] C.-Y. Lin, *Rouge: A package for automatic evaluation of summaries*, Jul. 2004. Accessed: Dec. 8, 2025. [Online]. Available: `https://aclanthology.org/W04-1013.pdf`

[34] Y. Liu, G. Tilahun, X. Gao, Q. Wen, and M. Gervers, *A comparative study of static and contextual embeddings for analyzing semantic changes in medieval latin charters*, 2025. Accessed: Oct. 12, 2025. [Online]. Available: `https://aclanthology.org/2025.loreslm-1.14.pdf`

[35] D. Machalica and M. Matyjewski, "Cad models clustering with machine learning," *Archive of Mechanical Engineering*, vol. 66, no. 2, pp. 133–152, Apr. 2019. DOI: `10.24425/ame.2019.128441` Accessed: Oct. 25, 2025. [Online]. Available: `https://yadda.icm.edu.pl/baztech/element/bwmeta1.element.baztech-77666cdd-ada0-472e-b608-86fafaa40164`

[36] L. Mandelli and S. Berretti, *Cad 3d model classification by graph neural networks: A new approach based on step format*, arXiv.org, Oct. 2022. Accessed: Oct. 13, 2025. [Online]. Available: `https://arxiv.org/abs/2210.16815`

[37] L. Mason, "Data vs information," *www.academia.edu*, Jan. 2019. [Online]. Available: `https://www.academia.edu/39505535/Data_vs_Information`

[38] D. Maturana and S. Scherer, "Voxnet: A 3d convolutional neural network for real-time object recognition," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2015, pp. 922–928. DOI: `10.1109/IROS.2015.7353481`

[39] J. D. McGlothlin, "Health hazard evaluation report: Hhe-80-248-791, national oceanic and atmospheric administration, u.s. dept of commerce, washington, d.c.," *Health Hazard Evaluation Reports (HHE)*, pp. 1–10, Jan. 1981. DOI: `10.26616/nioshhhe80248791` Accessed: Nov. 28, 2025. [Online]. Available: `https://www.cdc.gov/niosh/hhe/reports/pdfs/80-248-791.pdf`

[40] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part i: Representation," *Computer-Aided Design*, vol. 35, pp. 83–94, Jan. 2003. DOI: `10.1016/s0010-4485(01)00177-4` Accessed: Oct. 25, 2025.

[41] M. El-Mehalawi and R. Allen Miller, "A database system of mechanical components based on geometric and topological similarity. part ii: Indexing, retrieval, matching, and similarity assessment," *Computer-Aided Design*, vol. 35, pp. 95–105, Jan. 2003. DOI: `10.1016/s0010-4485(01)00178-6` Accessed: Oct. 25, 2025.

[42] U. Muaz, *Invariance, causality, and robust deep learning*, Medium, May 2021. Accessed: Dec. 9, 2025. [Online]. Available: `https://medium.com/data-science/invariance-causality-and-robust-deeplearning-df8db9091627`

[43] W. Newhouse, M. Souppaya, J. Kent, K. Sandlin, and K. Scarfone, "Data classification concepts and considerations for improving data protection," *Data Classification Concepts and Considerations for Improving Data Protection*, Nov. 2023. DOI: 10.6028/nist.ir.8496.ipd [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ir/2023/NIST.IR.8496.ipd.pdf

[44] H.-N. Nguyen and L. Cuong, "Overview of data classification and applications in data security," *International Journal of Environmental Sciences*, vol. 11, pp. 31–39, Jun. 2025. DOI: 10.64252/ac0b3685 Accessed: Jul. 20, 2025.

[45] J. O'Donnell and C. Crownhart, *We did the math on ai's energy footprint. here's the story you haven't heard.* MIT Technology Review, May 2025. Accessed: Nov. 28, 2025. [Online]. Available: https://www.technologyreview.com/2025/05/20/1116327/ai-energy-usage-climate-footprint-big-tech/

[46] K. Olson, "What are data?" *Qualitative Health Research*, vol. 31, no. 8, 2021, Art. no. 1015960. DOI: 10.1177/10497323211015960

[47] O. Omolayo and O. Babatope, "Comparative evaluation of vector embedding frameworks for scalable sematic retrieval in pdf-based rag systems," *International Journal of Research Publication and Reviews*, vol. 6, pp. 12 506–12 511, Jun. 2025. DOI: 10.55248/gengpi.6.0625.23100

[48] P. Pajo, "Vector embeddings unveiled: A comprehensive exploration of their creation, types, applications,...," *ResearchGate*, Apr. 2025. DOI: 10.13140/RG.2.2.15544.05129 [Online]. Available: https://www.researchgate.net/publication/390598346_Vector_Embeddings_Unveiled_A_Comprehensive_Exploration_of_Their_Creation_Types_Applications_Challenges_and_Future_Directions_in_Machine_Learning/

[49] P. Papadakis, I. Pratikakis, T. Theoharis, and S. Perantonis, "Panorama: A 3d shape descriptor based on panoramic views for unsupervised 3d object retrieval," *International Journal of Computer Vision*, vol. 89, pp. 177–192, Aug. 2009. DOI: 10.1007/s11263-009-0281-6 Accessed: Oct. 7, 2025.

[50] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, Philadelphia, Pennsylvania, USA: Association for Computational Linguistics, Jul. 2002, pp. 311–318. DOI: 10.3115/1073083.1073135 Accessed: Dec. 8, 2025. [Online]. Available: https://aclanthology.org/P02-1040

[51] F. Parmeggiani, "Fear of the dark: Diazo printing by photochemical decomposition of aryldiazonium tetrafluoroborates," *Journal of Chemical Education*, vol. 91, pp. 692–695, Feb. 2014. DOI: 10.1021/ed400555a Accessed: Jul. 24, 2025.

[52] R. Patil, S. Boit, V. Gudivada, and J. Nandigam, "A survey of text representation and embedding techniques in nlp," *IEEE Access*, vol. 11, pp. 36 120–36 146, 2023. DOI: 10.1109/access.2023.3266377

[53] M. J. Pratt, "Introduction to iso 10303—the step standard for product data exchange," *Journal of Computing and Information Science in Engineering*, vol. 1, pp. 102–103, Jan. 2001. DOI: 10.1115/1.1354995 Accessed: Oct. 4, 2025.

[54] W. Qader, M. M. Ameen, and B. Ahmed, "An overview of bag of words;importance, implementation, applications, and challenges," pp. 200–204, Jun. 2019. DOI: 10.1109/IEC47844.2019.8950616

[55] S. Qaiser and R. Ali, "Text mining: Use of tf-idf to examine the relevance of words to documents," *International Journal of Computer Applications*, vol. 181, Jul. 2018. DOI: 10.5120/ijca2018917395

[56] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, *Pointnet: Deep learning on point sets for 3d classification and segmentation*, arXiv.org, 2016. [Online]. Available: https://arxiv.org/abs/1612.00593

[57] M. Rahman, M. Khatib, P. Rani, S. Shaikh, and G. Gunashekar, "Effect of computer aided drafting on manual drafting skills," *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, pp. 3012–3014, Sep. 2019. DOI: 10.35940/ijitee.K2315.0981119

[58] R. Roj, M. Sommer, H.-B. Woyand, R. Theiß, and P. Dueltgen, "Classification of cad-models based on graph structures and machine learning," *Computer-Aided Design and Applications*, vol. 19, pp. 449–469, Sep. 2021. DOI: 10.14733/cadaps.2022.449-469

[59] I. Sadrehaghighi, "Computer aided design (cad)," Jan. 2022. DOI: 10.13140/RG.2.2.12634.62408

[60] A. I. J. I. B. W. Samuels, "One-hot encoding and two-hot encoding: An introduction," Jan. 2024. DOI: 10.13140/RG.2.2.21459.76327

[61] J. Schoonenboom, "The fundamental difference between qualitative and quantitative data in mixed methods research," *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, vol. 24, Jan. 2023. DOI: http://dx.doi.org/10.17169/fqs-24.1.3986 Accessed: Oct. 13, 2025. [Online]. Available: https://www.qualitative-research.net/index.php/fqs/article/view/3986/4916

[62] K. Sfikas, I. Pratikakis, and T. Theoharis, "Ensemble of panorama-based convolutional neural networks for 3d model classification and retrieval," *Computers & Graphics*, vol. 71, pp. 208–218, 2018, ISSN: 0097-8493. DOI: https://doi.org/10.1016/j.cag.2017.12.001 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0097849317301978

[63] K. Sfikas, T. Theoharis, and I. Pratikakis, "Exploiting the PANORAMA Representation for Convolutional Neural Network Classification and Retrieval," in *Eurographics Workshop on 3D Object Retrieval*, I. Pratikakis, F. Dupont, and M. Ovsjanikov, Eds., The Eurographics Association, 2017, ISBN: 978-3-03868-030-7. DOI: 10.2312/3dor.20171045

[64] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132 306, Mar. 2020. DOI: 10.1016/j.physd.2019.132306

[65] S. K. Shivayogi, "Data classification methodologies and implementation," *Journal of Computer Science and Technology Studies*, vol. 7, pp. 202–210, May 2025. DOI: 10.32996/jcsts.2025.7.5.26 Accessed: May 31, 2025.

[66] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," *CoRR*, vol. abs/2104.09864, Nov. 2023. DOI: https://doi.org/10.48550/arXiv.2104.09864 [Online]. Available: https://arxiv.org/abs/2104.09864

[67] I. E. Sutherland, "Sketch pad a man-machine graphical communication system," in *Proceedings of the SHARE design automation workshop*, 1964, pp. 6–329.

[68] B. Suyunu, E. Taylan, and A. Özgür, *Linguistic laws meet protein sequences: A comparative analysis of subword tokenization methods*, 2024. arXiv: 2411.17669 [cs.CL]. [Online]. Available: https://arxiv.org/abs/2411.17669

[69] M. Szilvsi-Nagy and G. Mátyási, "Analysis of stl files," *Mathematical and Computer Modelling*, vol. 38, Oct. 2003. DOI: 10.1016/S0895-7177(03)90079-3

[70] T. Taipalus, "Vector database management systems: Fundamental concepts, use-cases, and current challenges," *Cognitive Systems Research*, vol. 85, p. 101 216, 2024, ISSN: 1389-0417. DOI: https://doi.org/10.1016/j.cogsys.2024.101216 [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389041724000093

[71] P. Taylor, *Data growth worldwide 2010-2028*, Statista, Jun. 2025. Accessed: Oct. 12, 2025. [Online]. Available: https://www.statista.com/statistics/871513/worldwide-data-created/?srsltid=AfmBOoos3sZoZKw6aS7wf2CiXFwWvkUGV6VUzUBHutfpJE82JTtKg53o

[72] C. Trabelsi et al., "Deep complex networks," *CoRR*, vol. abs/1705.09792, 2017. DOI: https://doi.org/10.48550/arXiv.1705.09792 Accessed: Dec. 8, 2025. [Online]. Available: https://arxiv.org/abs/1705.09792

[73] H. Tran, *Natural Language Processing for Everyone*. Bookdown, 2024. [Online]. Available: https://bookdown.org/tranhungydhcm/mybook/

[74] G. V. Trunk, "A problem of dimensionality: A simple example," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, pp. 306–307, Jul. 1979. DOI: 10.1109/TPAMI.1979.4766926 Accessed: Dec. 29, 2021. [Online]. Available: https://ieeexplore.ieee.org/document/4766926

[75] A. Vaswani et al., "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17, Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010, ISBN: 9781510860964.

[76] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, *Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned*, 2019. Accessed: Oct. 12, 2025. [Online]. Available: https://aclanthology.org/P19-1580.pdf

[77] D. Vranic, “3d model retrieval,” in *PhD Thesis, University of Leipzig*, Section 3.2: Pose Normalization, 2003.

[78] B. Wang, A. Wang, F. Chen, Y. Wang, and C.-C. Kuo, “Evaluating word embedding models: Methods and experimental results,” *APSIPA Transactions on Signal and Information Processing*, vol. 8, Jan. 2019. DOI: `10.1017/ATSIP.2019.12`

[79] R. Wu, C. Xiao, and C. Zheng, “Deepcad: A deep generative network for computer-aided design models,” *arXiv.org*, Aug. 2021. DOI: `https://doi.org/10.48550/arXiv.2105.09492` Accessed: Dec. 8, 2025. [Online]. Available: `https://arxiv.org/abs/2105.09492`

[80] Y. Wu et al., *Google’s neural machine translation system: Bridging the gap between human and machine translation*, arXiv.org, 2016. Accessed: Dec. 29, 2025. [Online]. Available: `https://arxiv.org/abs/1609.08144`

[81] C. You, H. Dai, Y. Min, J. S. Sekhon, S. Joshi, and J. S. Duncan, “Uncovering memorization effect in the presence of spurious correlations,” *Nature Communications*, vol. 16, Jul. 2025. DOI: `10.1038/s41467-025-61531-5` Accessed: Jul. 24, 2025. [Online]. Available: `https://www.nature.com/articles/s41467-025-61531-5`

[82] I. Zeid, *Mastering Solidworks*. Macromedia Press, 2021.

[83] *Understanding deep learning requires rethinking generalization*, ICLR 2017, Oct. 2025. Accessed: Dec. 8, 2025. [Online]. Available: `https://openreview.net/forum?id=Sy8gdB9xx`

[84] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “Bertscore: Evaluating text generation with bert,” *CoRR*, vol. abs/1904.09675, Feb. 2020. DOI: `https://doi.org/10.48550/arXiv.1904.09675` Accessed: Dec. 9, 2025. [Online]. Available: `http://arxiv.org/abs/1904.09675`

[85] C. Zhou et al., “Lima: Less is more for alignment,” *arXiv (Cornell University)*, May 2023. DOI: `10.48550/arxiv.2305.11206` Accessed: Apr. 24, 2026. [Online]. Available: `https://arxiv.org/pdf/2305.11206`